InpaintAR

A Comparison of different InPainting Approaches for Real-Time Use

Sebastian Gradwohl

BACHELOR THESIS

submitted to the
Bachelor's Degree Program
Software Engineering

at the
University of Applied Sciences Upper Austria
in Hagenberg

2026

Supervisor: FH-Prof. Dr. Christoph Anthes, MSc

Declaration

I hereby declare and confirm that this thesis is entirely the result of my own original work. Where other sources of information have been used, they have been indicated as such and properly acknowledged. I further declare that this or similar work has not been submitted for credit elsewhere. This printed copy is identical to the submitted electronic version.

Hagenberg, February 28, 2026

Sebastian Gradwohl

Contents

Abstract

Augmented Reality applications are gradually becoming more popular. In Augmented Reality, the real environment is shown to the user, enhanced through additional virtual objects.

In recent years, there have been many technological advances in both hardware and software-integration, like the growing support for hand-tracking, inside-out tracking, advancements in Head Mounted Displays in terms of performance and camera quality. This is assisted by the growth of the market through recent hardware releases, like the Apple Vision Pro[1], and the Samsung Galaxy XR[2] headset.

Augmented Reality is very useful for use cases like immersive analytics, in which the users can interact and view data in intuitive ways. However, for this use-case, the user's focus should be on the visual data analysis. In such a scenario, visual clutter, which can be caused by both physical and virtual objects, may prove disadvantageous and reduce the benefits provided. Whilst the virtual objects can be managed inside an application, physical objects pose a problem.

Therefore, this bachelor's thesis focuses on the analysis of the different methods and algorithms for the concealment of the clutter to provide a clean and non-distracting environment for the digital object to be displayed on.
To accomplish this, a prototype to select planes on which objects are detected to subsequently be hidden through inpainting will be implemented for the Unity3D Game Engine. Through the usage of this implementation, several methods and algorithms will be evaluated in terms of both quality and performance to provide believable and uncluttered environments, whilst at the same time minimizing the use of computational resources.
These findings are compared to each other to find both the advantages and disadvantages each of the analyzed approaches delivers.

[1] https://www.apple.com/apple-vision-pro/
[2] https://www.samsung.com/us/xr/galaxy-xr/galaxy-xr/

Kurzfassung

Augmented-Reality-Anwendungen werden immer beliebter. In Augmented Reality wird dem Nutzer die reale Umgebung angezeigt, welche durch zusätzliche virtuelle Objekte erweitert wird.

In den letzten Jahren gab es viele technologische Fortschritte, sowohl bei der Hardware- als auch bei der Software-Integration, wie zum Beispiel die zunehmende Unterstützung von Hand-Tracking, Inside-Out-Tracking und Fortschritte bei Head-Mounted-Displays in Bezug auf Leistung und Kameraqualität. Dies wird durch das Wachstum des Marktes aufgrund der jüngsten Hardware-Veröffentlichungen unterstützt, wie beispielsweise die Apple Vision Pro[3] und das Samsung Galaxy XR Headset[4].

Augmented Reality ist sehr nützlich für Anwendungsfälle wie immersive Analysen, bei denen die Benutzer auf intuitive Weise Daten anzeigen und mit diesen interagieren können. Bei diesem Anwendungsfall sollte sich der Benutzer jedoch auf die visuelle Datenanalyse konzentrieren. In einem solchen Szenario kann visuelle Unordnung, die sowohl durch physische als auch durch virtuelle Objekte verursacht werden kann, sich als nachteilig erweisen und die Vorteile mindern. Während die virtuellen Objekte innerhalb einer Anwendung verwaltet werden können, stellen physische Objekte ein Problem dar.

Daher konzentriert sich diese Bachelorarbeit auf die Analyse der verschiedenen Methoden und Algorithmen zur Verdeckung der Unordnung, um eine saubere und ablenkungsfreie Umgebung für die Anzeige des digitalen Objekts zu schaffen.
Um dies zu erreichen, wird ein Prototyp zur Auswahl von Flächen, auf denen Objekte erkannt werden, die anschließend durch Inpainting ausgeblendet werden sollen, für die Unity3D Game Engine implementiert. Mithilfe dieser Implementierung werden verschiedene Methoden und Algorithmen hinsichtlich ihrer Qualität und Leistung bewertet, um glaubwürdige und übersichtliche Umgebungen zu schaffen und gleichzeitig den Einsatz von Rechenressourcen zu minimieren.
Diese Ergebnisse werden miteinander verglichen, um die Vor- und Nachteile der einzelnen analysierten Ansätze zu ermitteln.

[3] https://www.apple.com/apple-vision-pro/
[4] https://www.samsung.com/us/xr/galaxy-xr/galaxy-xr/

Chapter 1

Introduction

The term Augmented Reality (AR) can be described as "augmenting natural feedback to the operator with simulated cues" [Mil+95]. This includes not only Head-Mounted-Displays (HMDs), for which the term has been commonly used, but also monitor and other screen based systems, in which computer-generated images are overlaid onto live or stored videos [Mil+95]. While many still consider AR a niche, the user numbers tell a different story. In 2023, Statista reported about 983 million users to be using AR on their mobile devices [Sta24]. This coincides with the release of many new devices, like the Meta Quest 3[1] and the Apple Vision Pro[2], current generation HMDs which support AR beyond mainstream mobile devices.

AR applications are, due to their nature of adding additional objects onto the user's viewing area, very prone to information overload. Whenever users are supposed to focus on the virtual objects at hand, the presence of physical objects may serve as a distraction [Che+22].
An existing solution is Diminished Reality (DR). This concept can be interpreted as a specific use case of AR, which aims to hide or remove certain physical objects from the viewport in real-time [MF02] (see figure 1.1). By adopting an approach similar to this thesis, DR may be used to reduce the visual clutter caused by the real environment.

To achieve this, the visual clutter from the viewport is intentionally drawn over for each frame. Whilst a fixed color rectangle would prove to be the simplest and most performant overlay for the clutter, it might cause the user to be even more distracted. Therefore, this thesis uses image inpainting, which was first pioneered by Efros and Leung as a solution to generate larger textures from a source texture and fill holes in images [EL99], to produce a plausible and believable substitute for the overlaid area.

1.1 Motivation

Clutter is an ever-present phenomenon in our daily lives. It is rather difficult to automatically detect visual clutter, as there is still a lack of definite understanding of which factors, features and attributes in objects are relevant and affect a user's attention span negatively [RLN07]. It has been shown through multiple experiments and studies that

[1] https://www.meta.com/en/quest/quest-3/
[2] https://www.apple.com/apple-vision-pro/

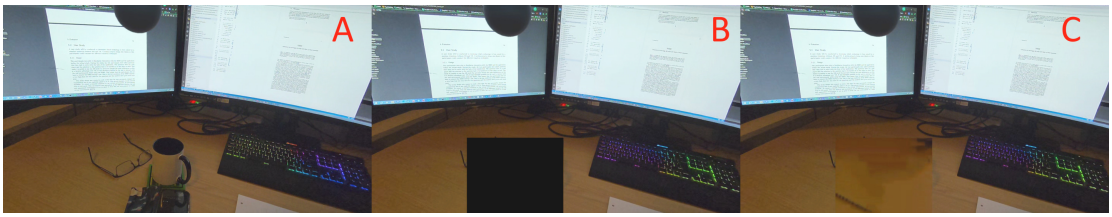


Figure 1.1: Real and diminished scene. This is an example of how DR may be used to reduce visual distractions. (A) shows the regular environment, (B) shows the same image with the visual clutter overdrawn with a mask, whilst (C) shows the cleaned up DR image, achieved through inpainting.

reducing visual distractions such as clutter improves a person's ability to focus and keep a longer attention span, see for example [MA23].

Over the last years, there have been several solutions for reducing distractions caused by real-world interference, most of them for sound. Whilst there are many solutions available for sound related distractions, like noise-cancellation headphones, at the moment, there is very little awareness of the distraction caused by visual noise, not to mention a severe lack of universal solutions for it [Hon+24]. During an experiment on which visual factors might impact focus, next to lighting and motions, information overload, as well as distractions caused by objects such as food, animals, large textures, or bright spots have been pointed out by the participants [Hon+24].

There are already a few existing solutions for achieving DR using inpainting, as seen for example in "DeepDR", a project to provide structure-aware Red-Blue-Green and Depth (RGB-D) aware inpainting for DR [Gsa+24], the evaluation of which of the existing inpainting methods is viable and could be used for not only this domain, but inpainting in a 3-dimensional real-time environment in general has largely been disregarded as of now.

Most publications put the focus on different components of the system, like the detection of objects ([Ram+16]), depth detection ([Gsa+24]), user studies ([Che+22]) or optimizations of smaller visual artifacts ([KSY16]). At most, there are some evaluations for comparison when presenting new state of the art algorithms and solutions for inpainting ([DRR19], [CJL23]), but an overview of existing algorithms and solutions and the evaluation in a real-time DR setting has been missing as of now.

The main goal of this analysis is therefore to provide an easy-to-reference comparison of the different methodologies to be compared in terms of both performance and quality in a real-time setting. Through this analysis, developers can make an informed choice about which methodology to use for their implementations, be it high fidelity in use cases like PC-powered AR systems or low energy, low-performance systems, more suited for mobile devices like mobile phones or standalone Mixed-Reality HMDs. In addition, the program built for this evaluation should be easy to extend for newer inpainting methods and swap out individual components.

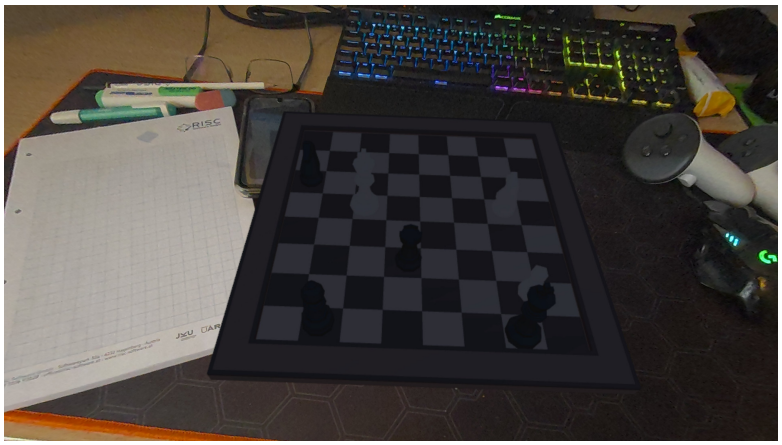


Figure 1.2: An example of virtual objects enhancing an already cluttered environment and therefore adding to the existing visual clutter

1.2 Overview

This work includes an introduction to the underlying concepts and analyzed inpainting methods, followed by the conceptualization and implementation of the presented solution as well as the performance and quality evaluation for each of the analyzed inpainting methods. These topics are elaborated upon in the following five chapters:

- **Chapter 2: Fundamentals and Related Work**
  Initially, an introduction into visual clutter and why it is of such high importance, explicitly to the domain of AR, is provided. The term of DR is explored in further detail, as well as the underlying logic of each of the analyzed inpainting methods.
- **Chapter 3: Concept**
  After providing a short overview of the requirements for the application, further details on the exact workflow for the prototype, as well as the design decisions for the inpainting and evaluation algorithms.
- **Chapter 4: Implementation**
  The hard and software requirements are presented, as well as an explanation of the software architecture, before presenting the implementation details of each of the components necessary for the workflow and evaluation, as well as a list of the remaining limitations in the implementation.
- **Chapter 5: Evaluation**
  The testing setup is presented, followed by the results for each test cases. Each test case is tested with every implemented algorithm and evaluated in terms of performance, quality and clutter reduction.
- **Chapter 6: Conclusion and Future Work**
  A brief summary of the results of the thesis is provided, followed by ideas on how the topic of inpainting in AR and visual clutter may be further analyzed.

Chapter 2

Fundamentals and Related Work

Before starting with the introduction to the implemented approach, some general knowledge and information needs to be further elaborated upon. All of the concepts, as well as which algorithms and methods are being leveraged to reduce the user's distraction through visual clutter, are being presented, starting from a more detailed explanation on visual clutter and why it is relevant in Mixed Reality (MR). This is followed by an introduction to Diminished Reality (DR), inpainting, how those two concepts can be combined, and the inpainting methods analyzed in this thesis.

2.1 Visual Clutter

Clutter can be seen as a state in which physical objects, their placement or some visual features cause performance degradations in certain tasks. This can range from many small objects, like a heap of puzzle pieces, all the way to single highly distracting objects, like lava lamps. It can lead to a decrease in object recognition performance or cause issues when searching for certain objects. In addition, clutter also seems to have a detrimental effect to short-term memory, whereas not only the number of elements, but their features can have a negative effect as well [RLN07].

Visual Clutter in Mixed Reality

Mixed Reality (MR), as defined by Paul Milgram in 1994, can be seen as an "environment in which real and virtual objects are presented together through a single display". Therefore, AR, DR and Virtual Reality (VR) can all be seen as subcategories of MR, which are placed somewhere in the virtuality continuum. This continuum spans from a completely real environment to a completely virtual one [MK94].

As defined by Mann and Fung back in 2002, AR aims to enhance reality through the addition of virtual objects, whilst DR aims to remove physical objects from the user's perception [MF02].
During the design of traditional software and information visualization, information overload and clear user interfaces need to be carefully considered. In these domains, all of the factors which may induce clutter and information overload can be controlled internally in the visualization or software [RLN07].

AR is uniquely flawed in that regard because it enhances the user's surrounding

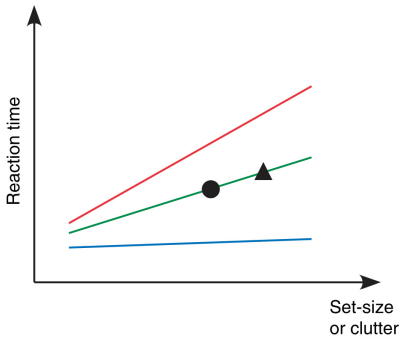


Figure 2.1: Image taken from [RLN07], Page 3; It shows a comparison between reaction time and clutter to predict performance. The difficulty of the given task may also change the increase in reaction time compared to the clutter, therefore 3 curves are provided instead of one.

environment. This in turn means there could be several external factors which may cause clutter, starting from the pure concept and interactivity of AR oftentimes allowing objects to be placed freely in the environment and therefore cluttered environments, to the space constraints of the user themselves.

2.2 Diminished Reality

DR was first introduced as a subsection of MR by Mann and Fung [MF02], often also called Mediated Reality (see [Man02], [GG+03] and [Poe+12]), due to the concept of software mediating both the physical and virtual objects the user is allowed to see. DR aims to remove or diminish real-world objects from the perception of the user by hiding them from their view. This can be accomplished by either completely hiding the objects or making them slightly transparent, so the user can still recognize the object when looking directly at it, however they perceive its presence less when looking at different objects [Che+22].

There have already been many experiments using DR, ranging from visually removing the front vehicle, which is being overtaken from a driver's viewport for safety improvements during overtaking [Ram+16], to interior design previews and furniture placement previews, as seen in [Sil17].
These works use similar concepts to those implemented for this thesis, working with inpainting algorithms to hide physical objects for the user's benefit, be it decision making for interior designs, increasing driver safety during overtaking, or, in the case of this thesis, in reducing distractions caused by clutter.

There have also been user studies to experiment and find out which effects are the most comfortable and effective for users, ranging from simply reducing the opacity of objects, to outlining, blurring or desaturating the unwanted objects. According to the experiments by Cheng et. al. in [Che+22], opacity reduction and outlining provide the best results. This can be interpreted to mean the less a user notices objects, the

less they are distracted by them. In addition, another experiment measured how users react to being able to decide which objects should be hidden compared to every object being detected and hidden automatically. The findings of this experiment have shown a preference for the users to decide for themselves which objects should be hidden and which should stay [Che+22].

These findings were taken into account during the planning of the implementation developed in the following chapters, as to how objects should be hidden and allowing for the user to select the areas and therefore the objects they want to hide for themselves instead of the software deciding.

2.3 Inpainting

Whilst DR itself is a great concept for the use case of reducing clutter, the problem remains of what to overlay over the objects to hide them. A single color block would prove to be the easiest solution, but it would ruin the believability of the scene for the user and could prove to be even more distracting than the objects which were in the covered area beforehand. Therefore, this overlay needs to be filled with a plausible background. Inpainting provides a good solution to fill these regions of the viewport and create believable overlays.

Definition

Image inpainting is based on the concept of recovering and completing missing regions in an image or removing objects from it. This concept has been around for many centuries, all the way back to the Renaissance, having been used in image restoration or removing damage caused by deterioration [Gil06]. In recent years, it has gained more popularity due to the improvement of image processing methodologies and the gradual adoption of digital image editing. To automate the inpainting process, several methods have been developed to create such an effect on an area of the users choosing without any further input. These range from static sequential algorithms over convolutional neural network (CNN) based approaches, using neural networks with automatic deep learning, up to generative adversarial network (GAN) based solutions, which train specialized inpainting models using a large number of provided images [Elh+20].

These categories and many of the provided methods provide adequate results for the inpainting of images. DR requires these methods to perform the inpainting as quickly as possible to adjust to the user's movement and provide believable results. This factor is especially relevant when using HMDs, as the micro-movements made by the user are already sufficient to displace the area of interest by several pixels.

Therefore, there are several methods specifically for the use in real-time inpainting. Some examples are the usage of edge-computing to outsource the inpainting work to a separate server, as seen in [Zha+22] or to improve the inpainting process through concepts such as feature re-use and mask propagation, as seen in [Liu+25].

Evaluated inpainting methodologies

As there are many different methods currently available regarding inpainting, analyzing all of them would require a lot of time and would quickly become outdated. Therefore,

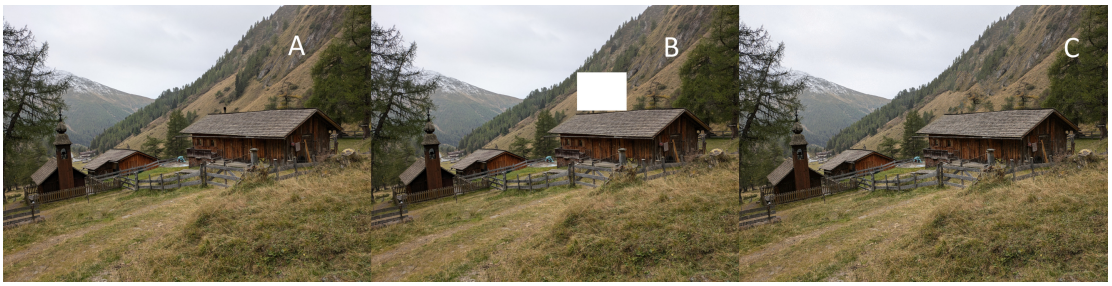


Figure 2.2: An example of inpainting, (a) original image, (b) image with synthetic damage, (c) image fixed with texture synthesis of surrounding area pixels

only a few candidates which seem promising for DR are being looked at in detail to provide an overview.

From gaining an understanding, which categories correlate to certain desired benefits, readers may get an overview of how other methodologies from the same category might perform. Of these methodologies, the following have been analyzed:

Fast Marching Method

Having first been proposed back in 1999 by Sethian, see [Set99], the Fast Marching Method (FMM) is an analytical method, which was first developed to solve boundary problems.

FMM has been proven by Telea to also be applicable to pixel-based problems, like inpainting. The algorithm works by creating an image smoothness estimator, which estimates the image smoothness of a given neighborhood to fill. These are used as a level set for the FMM algorithm to propagate the image information [Tel04].

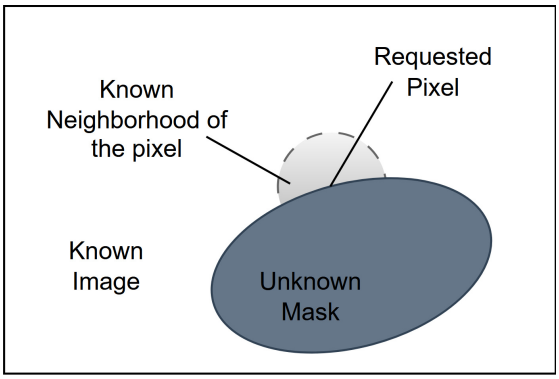


Figure 2.3: Concept of the Fast-Marching Inpainting

The mathematical formula for the calculation of a single pixel's value is defined as:

$$I(p) = \frac{\sum_{q \in B\epsilon(p)} w(p,q)[I(q) + \Delta I(q)(p-q)]}{\sum_{q \in B\epsilon(p)} w(p,q)}$$

By calculating the central differences of all the known neighboring pixels (q) in correlation to the target pixel (p), an image gradient may be estimated, which is in this case displayed as ΔI.
$B\epsilon$ in this case describes the known area around the pixel, from which the synthesized pixel should be generated. To avoid any outlier pixel values from negatively influencing the pixel synthesis, these points are all summed up and weighted by a normalizing function w. This normalization weight function can be described with this formula:

$$w(p,q) = dir(p,q) \times dst(p,q) \times lev(p,q)$$

For this weighting function, there are three components which need to be considered:

- The direction (dir) of the incoming pixel in correlation to the target pixel. This is relevant to avoid information being lost and to keep structural integrity to some degree. Therefore, when moving inwards in the mask, pixel values, which came from the same direction, are prioritized over other surrounding values.
- The distance (dst) is relevant with regard to a smooth color gradient for the inpainted mask. The further the algorithm moves inwards, the more unstable the surrounding known neighborhood of the pixel becomes, due to containing more inpainted pixels with each layer. To avoid intensity artifacts, the values of pixels which are closer to the target pixel are prioritized over pixels which are further away.
- Lastly, the level set (lev) focuses on the distance of the pixels considering the contour of the mask in general, putting a greater weight on source pixels which are closer to the mask itself

whereas these three factors end up with the following mathematical formulas:

$$dir(p,q) = \frac{p-q}{||p-q||} \times N(p)$$

$$dst(p,q) = \frac{d_0^2}{||p-q||^2}$$

$$lev(p,q) = \frac{T_0}{1 + |T(p) - T(q)|}$$

d_0 and T_0 reference the relative interpixel distance, usually set to 1 [Tel04].
It was based on the model of manual inpainting heuristics first introduced by Bertalimo on how to paint a point by introducing colors from a small region around it [Ber+00].

This algorithm alone can only calculate a value for one color channel. By applying

it to all 3 channels of the RGB spectrum, it can easily be adjusted to calculate pixel values for colored video frames.

The advantages of the algorithm are its easy-to-implement nature, allowing for an easy understanding of the algorithm, as well as its efficiency and quality given its simplicity, whilst its disadvantage is the loss of structural information.

There have been proposals and solutions for this disadvantage for the FMM method, though. Sari, Horikawa and Du proposed a solution which segments large missing regions through structure propagation to split the missing region into multiple smaller missing image regions [SHD21]. During initial testing, the performance degradations of these quality improvements were too severe, which is why it was not implemented in the final prototype despite its quality improvements.

Nonlocal Texture Matching and Nonlinear Filtering

Proposed back in 2019, see [DRR19], this method aims to find existing neighboring areas, which match the existing surrounding texture to find a pixel's value. Whilst this approach is a lot more complex compared to the simple FMM, it promises higher quality results, due to working through patches of pixels and trying to keep a consistent pattern within them.

This approach is also known as exemplar-based inpainting and was established as a cheap and effective way to generate textures from known areas of an image to fill missing areas.

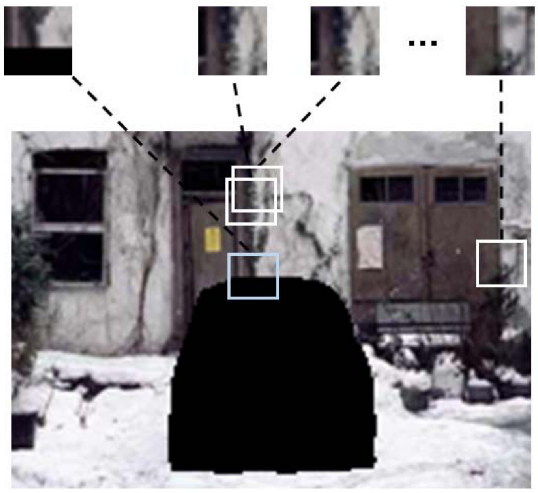


Figure 2.4: Taken from [DRR19], it shows an example of the candidate selection for the inpainting patch. The missing areas from the target patch are then filled with normalized and averaged values from the correlating candidate patch pixels.

It accomplishes this by first prioritizing the patches of the mask to be filled. This patch contains both a source region and the unknown part, in which the contour of the missing region is in the center of the patch, as can be seen in the upper left patch of figure 2.4.

The priority function is defined as:

$$P(\Psi_p) = C(\Psi_p) \times D(\Psi_p), p \in \delta\Omega$$

In this function, Ψ_p is defined as the selected patch, whereas the pixel p is defined as the center point of the patch, which is a part of the unknown area boundaries $\delta\Omega$. $C(\Psi_p)$ defines the confidence term, which is defined as the ratio of known pixels within the given patch.

The function D in the function above represents the data about the strength of the surrounding structural information, like edges and textures at the given pixel and how they pass through the missing region to propagate information for inpainting around these pixels, rather than across them.

$D(\Psi_p)$ is mathematically defined as the dot product of the isophote vector of p (ΔI_p). This vector represents the edges and textures around the target patch and can be interpreted as a vector with a 90° rotation compared to the gradient, therefore following the contours of the image, which can be recognized by lines of constant intensities. As the NLTM algorithm puts a large focus on keeping textures and edges consistent in the inpainted areas, this needs to be considered accordingly.

$$D(\Psi_p) = \frac{\Delta I_p \times n_p}{I_{max}}$$

$$\Delta I_p = \frac{1}{2I_{max}}\{[I(x_p, y_p - 1) - I(x_p, y_p + 1)] \times i + [I(x_p + 1, y_p) - I(x_p - 1, y_p)] \times j\}$$

The isomorph vector function shown above may seem complicated at first, but after breaking it down, it becomes easy to understand. The subtractions of the intensity, $I(x_p, y_p-1)-I(x_p, y_p+1)$ serves to estimate the intensity change in a vertical dimension, whilst the same subtraction with modifications to x serves to provide the intensity change of the horizontal dimension.

To actually create a vector, i and j are defined as unit vectors in the x and y direction, which are used to modify the vectors according to the intensities.

These values are then normalized using the term $\frac{1}{2I_{max}}$, which helps to compare the patches with each other, as the intensities may differ significantly between different parts of the mask contour surroundings. Through this function, the patches can be prioritized, by considering the existing contours within and comparing them with the contours and textures.

Since the isomorph vector already provides information about which direction the structures of the image wish to continue towards, it should also be considered in which direction the mask needs to be filled when looking at the center of the pixel patch. This is where the normal vector n_p comes in:

$$n_p = [M(x_p + 1, y_p) - M(x_p - 1, y_p)] \times i + [M(x_p, y_p + 1) - M(x_p, y_p - 1)] \times j$$

n_p in this case defines the normal vector at the center pixel p. M returns 1, in case the given coordinate in the parameters is inside the known pixels and 0 in case it is at

the position of an unknown pixel. This calculation ensures the filling progresses from the outer contour of the mask inwards, whilst also assuring a consistency for the edge continuation of the isomorph vector.

After the prioritized target patch is chosen, a non-local texture similarity (NLTS) measure is used to select candidate patches as sources for filling the target region. This NLTS is defined as such:

$$NLTS(I_p, I_q) = \frac{-||(I_p^2 - I_q^2)\ o\ G_p||1}{h^2}$$

o, in this case, is defined as the element-wise product operator, whilst h is a user-selected parameter, which should be proportionately larger than the maximum color value (usually 255). G_p is the Gaussian kernel, which is the weight of how different a patch is in terms of the pixel intensities, in correlation with the target patch with p at its center.

Using this NLTS measure, a number of source regions may be collected. The more candidates the following algorithm has, the less error-prone the inpainting.

After deciding on which candidate patches to pick, for each of the missing pixels, the mean value of all the candidates' corresponding pixels is calculated and applied to the target pixel.
To support color, the isophote vector, used for the priority calculation, is computed as the average of all three color channels. The NLTS needs to be modified as well to sum up the value above for each of the three color channels [DRR19].

Exemplar-Based Image Inpainting with modified priorities

This method was first proposed in 2015 by Deng et. al., see [DHZ15].
For this algorithm, many approaches are similar to the non-local texture matching, see chapter 2.3. It was chosen as an additional method, as it was shown to be more performant in the benchmarks in [DRR19], although the results were apparently slightly worse.

Such exemplar-based algorithms put the focus on the surrounding area of the target pixels, therefore reducing the search area for candidate pixel patches marginally and improving the performance, albeit at a potential loss of quality.

Compared to the approach of the prior algorithm, the priority function is overhauled to the following:

$$P(p) = \begin{cases} \mathrm{D(p)}, & \text{first phase} \\ \mathrm{C(p)}, & \text{second phase} \end{cases}$$

As for how many steps each phase has, the first phase needs to be estimated or set statically. Afterwards, the second phase may continue until the missing region is filled out completely.
To reduce the computational load of the similarity between two patches, the formula was changed accordingly:

$$\Psi_q = argmin\ d(\Psi_p, \Psi_q)$$

$d(\Psi_p, \Psi_q)$ in this case describes the sum of squared differences of the known pixels between the two patches.

After finding the patch with the highest similarity, it is used to fill the missing pixels in the target patch. This is one of the methodologies in which it differs from the NLTS approach of Ding, Ram, Rodriguez (see [DRR19]). Instead of picking multiple source patches and using a mean value, it only considers one [DHZ15].

This should therefore reduce the runtime of the process. In return, this will cause the method to be more susceptible to inconsistent candidate patches and for the output to potentially be of lower quality.

Large Mask Inpainting with Fourier Convolutions

Moving away from algorithmic methods, many modern methods also use trained inpainting networks to provide high-quality results. Whilst using inpainting models might seem slower than using algorithmic models, most of them provide higher quality images and may handle large masks more efficiently due to the large computational complexity in algorithmic solutions.

One of those networks is the Large Mask (LaMa) inpainting network, which uses Fourier Convolutions to inpaint large masks effectively [Suv+22]. Whenever large areas need to be filled using inpainting, the global context of images needs to be considered. The Fast Fourier Convolution (FFC), first introduced in 2020 by Chi [CJM20], is an operator which allows the use of global context in early layers of the inpainting pipeline. It manages this by splitting into a local and a global branch, evaluating those two branches before merging the results back together.

As networks are often trained on lower-resolution images, it is very common to first downscale the image, process it, and upscale it afterwards. LaMa uses a similar approach.

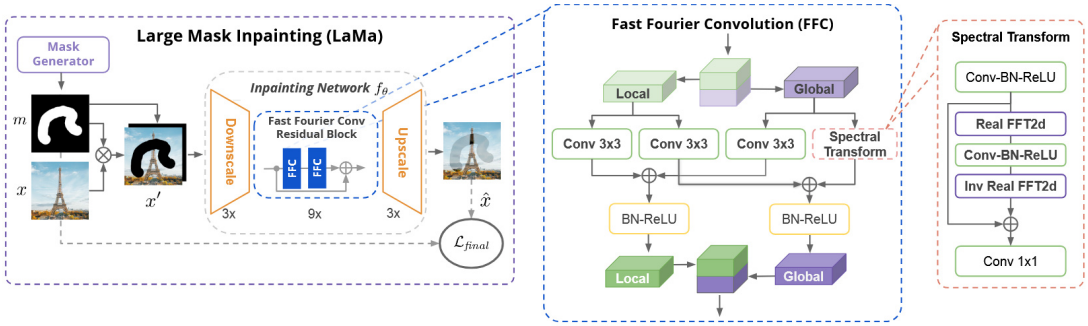


Figure 2.5: Taken from [Suv+22], the architecture of the LaMa model

In addition to the convolution, another large part of machine learning based inpainting models is the loss functions. The results generated by inpainting are inherently ambiguous due to the missing information of the inpainted background. This ambiguity only grows as the missing region becomes larger. For this, inpainting models implement multiple functions. In the case of LaMa, there are three functions, which are combined to lead to high-quality results.

- The first of the loss functions concerns itself with perceptual loss. This function concerns itself with the evaluation of the distance between different features, which are extracted from the predicted and target images by a pre-trained network.
- The second function puts its focus on the adversarial loss, focusing on generating naturally looking details. It accomplishes this by using a discriminator to differentiate between the existing and the generated patches. Thanks to the perceptual loss of the first function, this generator quickly learns to copy the existing areas of the image.
- In the final step, a gradient penalty is applied to stabilize the training data. It has also been proven to, in some cases, slightly improve performance [Suv+22].

Due to the existence of pre-trained models, the focus of this thesis lies on the integration and evaluation of the existing model inside a real-time inpainting process.

Expectations for Real-Time Usage

Based on the concepts of each of the analyzed approaches, it is highly likely for the FMM approach to be the most efficient, due to its relatively low computational cost; In return it should provide the lowest quality inpainting, due to simply stretching the pixels from the surroundings of the mask inwards. In stark contrast, NLTM should provide the highest quality inpainting, albeit at the cost of performance due to having to analyze the entire image and calculating the average of the candidate pixel values. In this regard, ELTM might provide a good trade-off, using the surrounding pixels similarly to FMM, but still analyzing pixel patches for similarity in a similar manner to NLTM
The LaMa Inpainting Model might prove to be very performant, due to being optimized for inpainting large masks efficiently. However, this assumes the inference of the model during the runtime does not require much overhead, which is unlikely. Depending on this overhead, the performance may be degraded significantly. Due to being trained on a very large dataset, the model should expectedly end up with the highest quality result.

All of the above arguments are summarized in table 2.1, in which the plausibility for real-time use based on these arguments is as follows:

Method	**Performance**	**Quality**	**Plausibility**
Fast Marching Method (FMM) [Tel04]	High	Low	High
Nonlocal Texture Matching and Nonlinear Filtering (NLTM) [DRR19]	Low	High	Low
Exemplar-Based Image Inpainting with modified priorities (ELTM) [DHZ15]	Medium	Medium	Medium
Large Mask Inpainting with Fourier Convolutions (LaMa) [Suv+22]	Medium	High	High

Table 2.1: Expected evaluation results considering the conceptual design

Chapter 3

Concept

In the previous chapter, several approaches and the underlying principles for DR were analyzed in detail. To analyze the aforementioned methodologies a evaluation program needs to be developed. This implementation will be created for an HMD, as it provides the user with a lot more freedom of movement and interactions compared to a phone camera.

In this chapter, the design decisions for the program are elaborated upon in further detail.

3.1 Requirements

The focus of this thesis lies on the evaluation of the inpainting methodologies and how they perform in a DR environment in terms of performance, quality and reduction of visual clutter. To achieve this, there are several surrounding factors which need to be addressed:

- The user should easily and intuitively be able to select the area to apply the inpainting to by themselves.
- It should be simple to switch inpainting methodologies on the fly. This stabilizes the evaluation results, as the mask and environment stays consistent.
- The evaluations of quality and clutter reduction should be calculated in the background to not affect the performance by blocking the main thread.
- The architecture should be modular and easy to extend, in case other methodologies need to be analyzed.

3.2 Workflow

The entire workflow of the prototype can easily be visualized in a flowchart (see figure 3.1) and was inspired by the work of Herling and Broll (see [HB10]), which made first advancements for object removal through inpainting in AR. As a baseline, a regular AR application in a Video-Based HMD always shows the camera feed in the background. This is most commonly achieved by using the camera feed for the skybox texture.

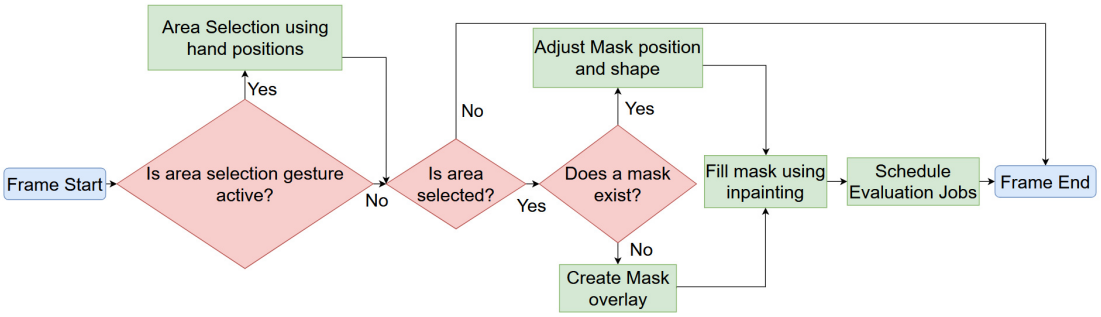


Figure 3.1: Flowchart visualization of the prototype's workflow

From this state, the user can start the workflow by selecting an area in their viewport, which is then used as the inpainting mask.

3.2.1 User Input

One of the considerations during the conceptualization of the implementation was the ease and simplification of the area selection.

Using Hand-Tracking to select the area should make it easy to pick up and use. Whilst hand gestures are easier to explain to users, it also avoids the requirement of having controllers to use the prototype, which would, of course, add additional visual clutter to the user's view.

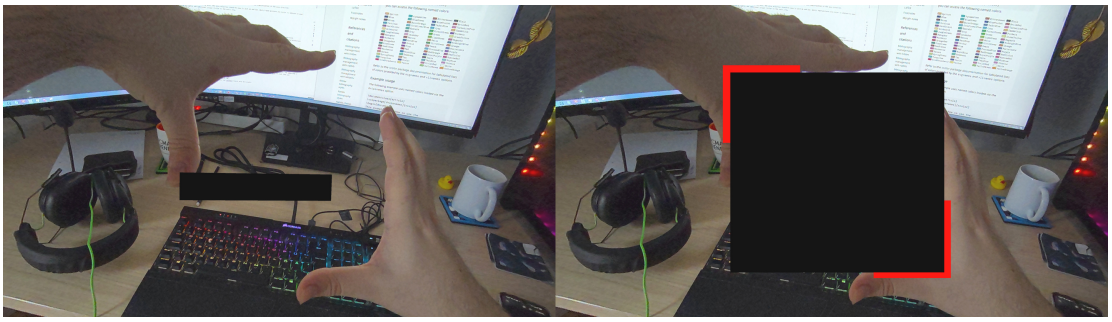


Figure 3.2: Area selection using hand gestures

By positioning their hands on the corners of the area they want to mask, the gesture should remain rather intuitive and easy to understand. Since the selection should only be triggered by a certain gesture instead of permanently being active, the L-gesture, as seen in figure 3.2, may allow the user to interpret their index finger and thumb as the contours of the mask. This gesture was shown to be useful for selecting a group of objects whilst allowing for easy resizing and repositioning of the mask with their hands back in 1997 by Pierce et. al. in their publication about image plane interactions for 3D environments [Pie+97].

The selected area can then be kept until the user selects a new area to overlay.

After the selection of the area to inpaint is finished, the constraints of the rectangle are transformed into concrete pixel coordinates on the inpainting overlay image.

3.2.2 Inpainting Design Decisions

A black box may prove distracting to the user, especially in brighter areas. This mask needs to be filled with plausible substitute pixels, which can be achieved through inpainting. Since these inpainting algorithms were designed for static images, there are a few design adjustments and optimizations to make them plausible for real-time use.

Whilst most of the methodologies analyzed in this thesis have vastly differing implementations, all of them can be boiled down to receiving an array of source pixels and a mask in the form of a list of the pixels to inpaint.

Since the methodologies analyzed in this thesis have only been conceptualized for static images, there are several optimizations to improve the performance for real-time use. Most of the methodologies need to iterate over a large number of pixels. To even make inpainting in a reasonable timespan plausible, an easy solution is to downscale the source image for the inpainting and upscale it again afterwards. Whilst this reduces the quality of the inpainting methodologies, it should, in return, end up improving the performance of the inpainting significantly.

Even with this reduction of the image size, the NLTM algorithm still takes much performance for its candidate-search. For this, the algorithm iterates over every pixel and calculates the similarity to the target pixel patch. Whilst this is intuitive and great if the focus is on quality rather than performance, it makes the algorithm infeasible for real-time use.

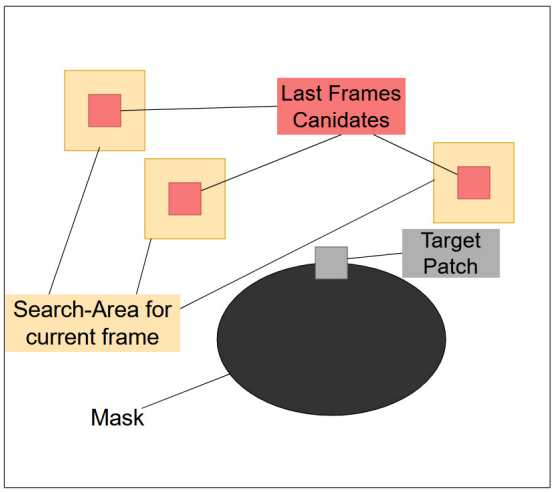


Figure 3.3: Prior frames candidates are used to reduce the candidate search area decisively

In theory, this candidate patch search over the entire image should only be necessary once, though. Since the viewport can only change up to a certain degree per frame,

instead of always searching the entire image again, the initial candidates can be re-used. This would mean that NLTM would use an approach similar to the ELTM algorithm or the PatchMatch algorithm of Barnes, Shechtman, Finkelstein and Goldman from 2009 (see [Bar+09]), albeit looking for candidates in the surrounding areas of the prior frames candidates instead of right next to the target pixel patch. As one can see in figure 3.3, this approach reduces the search area for inpainting candidates decisively and should provide a noticeable performance improvement with little to no quality loss in all subsequent frames after the initial full-image search frame.

3.3 Evaluation Approach

The evaluation of the algorithms needs to be fully automated and happen during the runtime. The presented algorithms aid in providing objective and impartial metrics for each of the inpainting algorithms, to compare them to each other without subjective bias.

Measuring Quality for Inpainting

The process of evaluating the quality of the inpainted area in an objective manner without a reference image is difficult. Traditional image quality evaluations usually focus on the entire image, whilst inpainting is mostly applied to smaller localized areas of the image. In 2019, Liang, Li, Hu and Zhang proposed a new Image Inpainting Quality Assessment (IIQA) to assess the quality of an inpainted image without any additional data [LIA+19].

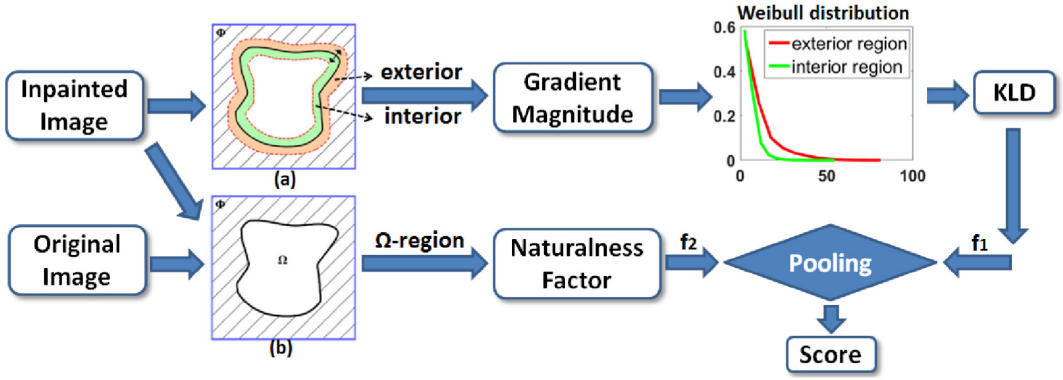


Figure 3.4: Taken from [LIA+19], flowchart of the IIQA metric

One factor for considering the inpainting quality is to look at the boundaries of the inpainted area. Both the outer boundaries with the original image contents, as well as the inpainted inner contents, are considered and compared for this. After calculating the gradient distributions, the Kullback-Leibler Divergence [KL51] is used to calculate the distance between the two distributions.

In addition to this, the naturalness of the original image and the inpainted image is

taken into account, which is based on a publication by Gong and Sbalzarini from 2015 (see [Gon+14]).

This calculation takes the gradients of the mask region and compares the weighted combination of the median value and the mean absolute deviation of the original image with the inpainted version.

By combining these two factors, both the local structure of the inpainted area, as well as the overall gradients, are considered when looking at the inpainting quality.

This approach has been shown to outperform other quality assessment metrics, as seen in [LIA+19]

Measuring Visual Clutter

As the reason for this implementation is to reduce clutter, it is important to calculate the reduction of clutter in the image thanks to the inpainting. This is relevant, as depending on the inpainting quality, the noise might increase to cause the area to become even more distracting than without inpainting.

For this, the approach suggested by Rosenholtz, Li and Nakano to measure the clutter by detecting unusual features and outliers inside of an image was used (see [RLN07]), albeit in a simplified manner. To accomplish this, the metric considers the color, contrast and the orientation of features inside an image:

- In terms of the color, the areas of the image are compared in terms of covariance and distribution. This means, if the pixels in a given area are close or similar in color, it is assumed to be perceived as little to no clutter. In case the covariance is high, the area might be distracting from a color standpoint and can be assumed to contain a lot of clutter. As the RGB Color-Space is not very well suited for perceived color values, the CIELab color space is used instead.
- For the contrast feature, the focus is on how high the variability of the contrast ends up. For this, the contrast of a pixel compared to its neighbors is calculated. These values for a given area are then used to calculate the standard deviation of the contrast, which defines how much features in a given space compete for the users attention.
- Lastly there is the orientation. This is in correlation to objects, which are pointing to different directions, seem more cluttered and distracting, rather than clearly aligned objects. This can be recognized, by looking at the Sobel gradient of the image and comparing the angle of edges accordingly. For these orientations, a coherence can be calculated, which can then be used to calculate the congestion.

In the publication, an approach of downscaling the image continuously was used [RLN07]. This continuously blurs out smaller features and distractions, leaving only the larger outliers left, which are then applied with a larger weight. Whilst this approach provides a better quality metric, the computational resources required are much higher and would slow down the evaluation to the point of only being able to consider a minority of the frames actually generated by the inpainting algorithm. Therefore, it was left out at the cost of a single scale evaluation approach, which causes the overall values to be slightly lower. As the clutter reduction of the inpainting approaches should be evaluated in correlation to each other, this should not be of consequence.

How Performance is measured

In terms of performance, the most common performance metric in real-time graphics applications has been proven to be Frames Per Second (FPS). This defines how many images can be iterated over in a single second and therefore stands in direct correlation to the calculation time required for the calculation of a single frame.

As the Total FPS can be affected by a lot of different external factors, like the mask preparation, receiving data from the camera and other secondary components, the time for inpainting is measured in an isolated manner as well. This can then be used to calculate an FPS metric, which only takes the time required for the inpainting algorithm itself into account. By analyzing and comparing both of these numbers, the performance of the inpainting algorithm itself and the additional overhead of the necessary preparations for inpainting an image are all taken into account.

Chapter 4

Implementation

This chapter presents the implementation of the prototype created for the evaluation. After providing an overview of the hardware and software used during the development, the different components of the program and the implementation details for the inpainting methods are explained in further detail.

Hardware

As the focus of this thesis lies on the evaluation of inpainting methodologies in an AR environment, an HMD which supports this technology, as well as provides access to the camera-data is required. In addition, the HMD should have the capabilities of hand-tracking for the area selection component.

A Meta Quest 3[1] is chosen for this, as it provides all the necessary features and was just recently extended with its new passthrough API[2], which allows for developers to directly access camera data. The HMD is equipped with two RGB cameras with 18 Pixels Per Degree for video see-through, whose output is displayed on two 2.064x2.208 pixel displays with a refresh rate of up to 120hz.

Whilst the entire implementation in itself is fully device-agnostic, therefore allowing any HMD which has the required features to work, some minor components, like the hand-tracking and controller input or the camera texture source, need to be modified according to the API of the required device.

Software

The implementation is built in the Unity3D Game Engine[3] in Version 6000.1.0f1.
It uses the Meta XR Core SDK[4] with Version 83.0.1 and the Meta MR Utility Kit [5] with a similar version for the integration of the Meta Quest 3 HMD and its APIs to the prototype.

The aforementioned Meta Passthrough API is built on top of the Android Camera2

[1] https://www.meta.com/en/quest/quest-3/
[2] https://developers.meta.com/horizon/documentation/spatial-sdk/spatial-sdk-pca-overview
[3] https://unity.com/products/unity-engine
[4] https://assetstore.unity.com/packages/tools/integration/meta-xr-core-sdk-269169
[5] https://assetstore.unity.com/packages/tools/integration/meta-mr-utility-kit-272450

API[6]. This allows for the camera to be used in a similar manner to mobile phone cameras, albeit with a few caveats:

- As of this moment, the passthrough API is not supported over Meta Quest Link. This means the software can only be evaluated on device, in the Meta Quest's standalone mode.
- The Camera Texture returned by the API is smaller than what a user sees in the Quest 3. Whilst there are extensions and improvements over time, as of this moment, it is not possible to cover the entire viewport of the user.
- The Camera Texture is inside the GPU memory. For inpainting and the quality evaluation, this needs to be sent to the CPU, which takes up additional latency [Met25].

As for the code itself, it is written in C# 9 on the .Net Standard 2.1. This version of the standard, in theory, supports all C# versions up to 10.0, but the version packaged with the Unity Game Engine only supports the language features up to C# 9.0 as of this moment [Tec26a].

4.1 Software Architecture

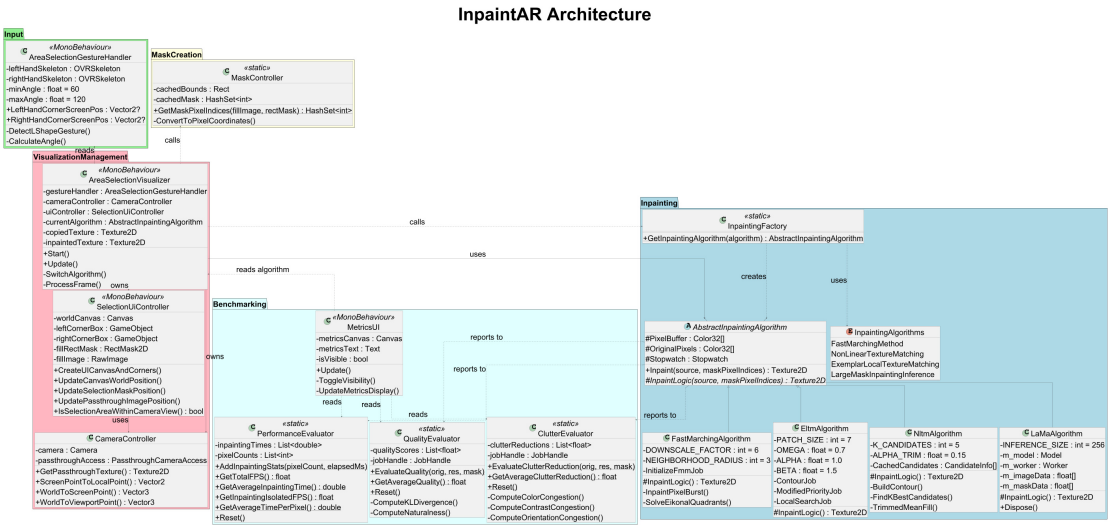


Figure 4.1: Simplified UML diagram of the implementation architecture

A large focus in the implementation is put on a modular approach in terms of architecture, as can be seen in figure 4.1. This allows for components in the implementation to be swapped out independently of each other. Therefore, in case this prototype gets extended and improved on in the future, for example with additional inpainting algorithms or different mask detection solutions, these components may be swapped out

[6] https://developer.android.com/reference/android/hardware/camera2/package-summary

without affecting other parts of the system.

In general, the entire prototype is split up into 3 larger namespaces with some helper classes to provide a well structured architecture with clearly split responsibilities:

- The inpainting namespace manages the entire inpainting logic and the inpainting algorithms themselves. These are separated in one AbstractInpaintingAlgorithm, which handles the function calls of the benchmarking classes, as well as the specific implementation classes. These classes merely implement the abstract protected InpaintLogic method, which is provided by the AbstractInpaintingAlgorithm class. Therefore, to trigger the inpainting algorithm, the virtual method Inpaint is called, which calls the InpaintLogic method internally. In addition to this, the namespace also contains the InpaintingAlgorithms enumerable, which is used to define which inpainting algorithm is currently selected and needs to be created by the InpaintingFactory. The factory class provides a simple static factory method for the creation of the AbstractInpaintingAlgorithm classes.
- The benchmarking namespace handles the evaluation metric implementations for performance, quality and clutter reduction. These classes are entirely static and provide public methods to both trigger the evaluation, as well as to get their results. The evaluations of each evaluator are done inside of separate background jobs. This allows for the evaluation to occur in the background to avoid them blocking the main thread and impairing performance. The results of these evaluators are fetched by the MetricsUI class, which can be bound to any Unity3D game object. It internally creates the UI elements to display all the necessary information for the evaluation results, which algorithm is active and the current framerate.
- Lastly, the visualization management namespace serves as the most important one. The AreaSelectionVisualizer class serves as the orchestrator of the entire system. This class triggers all the controllers in the prototype and manages them, starting from the mask creation at the hand positions provided by the AreaSelectionGestureHandler which is handled inside of the MaskController and SelectionUiController classes, to the inpainting algorithm function call. It also stores all the state variables, like the displayed inpainted texture, the selected algorithm and the different controllers. In terms of logic, the AreaSelectionVisualizer class checks if the selection area should be adjusted and if the inpainting mask is within the users viewport. If this is the case, the inpainting algorithm is called. It also handles the logic for switching algorithms by listening to the button press from a meta controller to cycle through them. In addition to this class, there is also the SelectionUiController, which handles all the logic concerning the canvas on which the inpainted texture is displayed, as well as the masks position. For this to position it at the correct position according to the passthrough camera, the CameraController helps to translate the 2D screen point of the camera texture to a specific world coordinate in the 3D space. This CameraController also manages both the user's camera, as well as the passthrough camera and is accessed to get the passthrough image texture. This texture is provided as a RenderTexture[7] from Meta's PassthroughCameraAccess class to use for inpainting.

[7] https://docs.unity3d.com/6000.3/Documentation/ScriptReference/RenderTexture.html

In addition to these namespaces, there is also the AreaSelectionGestureHandler class. This class is in its own Input namespace and handles all the logic concerning hand and gesture tracking in isolation. By constantly tracking the skeleton of both hands, as provided by the Meta XR Core SDK, the position of these skeletons can then be used to create a mask for inpainting.

In terms of performance measurements, the C# Stopwatch class is used track the elapsed time between the start of the Inpaint function and the completion of the inpainting algorithm, right before the evaluation scheduling. To avoid mixing up evaluation results, each evaluator class contains a reset method, which resets all the metric storage properties of the evaluator classes. This is mainly called after switching algorithms by the AreaSelectionVisualizer.

4.2 Area Selection and Tracking

As previously described in the concept chapter about user input (see 3.2.1), hand-tracking is used for defining the inpainting mask.

For this, the skeleton of the hand and the positions for each of the fingers' bones need to be tracked reliably. The Meta XR Core SDK already provides the tracking capabilities and mapping of the user's hand to a skeleton [Met26].

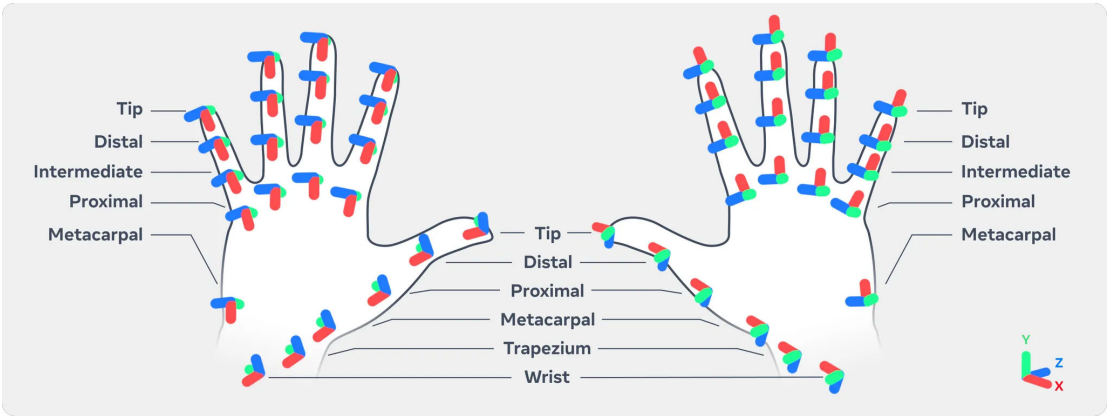


Figure 4.2: A visualization of the bones and joints tracked by the Meta XR Core SDK [Met24]

The tracked bones and joints, as seen in figure 4.2 each have their own unique transformation in the world. By using the rotation vectors of these transformations, the direction of the fingers and through these, the angle between two fingers can easily be calculated.

In the example shown in algorithm 4.1, the thumb and index finger are compared, based on the directional vector created when comparing the position of the knuckle or proximal with the tip of the finger. If they have an angle of close to 90° between each other, they create an L-like shape, which can then be used as the corner-point for the area selection. As the base-bone of the thumb is the metacarpal, the rectangle's corner

Algorithm 4.1: Gesture Recognition method

```
1: function CheckLShape(skeleton)
       Input: skeleton, the skeleton of a hand containing all the bone positions
       Returns if the shape has been recognized and the corner position
2:     cornerPos ← DefaultZeroVector
                                          ▷ Finger direction vector calculation
3:     indexVector ← Normalize(indexDistalPosition − IndexProximalPosition)
4:     thumbVector ← Normalize(thumbTipPosition − ThumbKnucklePosition)
5:     angle ← Angle(indexVector, thumbVector)
                          ▷ Allow tolerance around 90 degrees for better usability
6:     if 75 ≤ angle ≤ 105 then
7:         cornerPos ← thumbKnuckle
8:         return (true, cornerPos)
9:     return (false, cornerPos)
```

is slightly closer to the wrist than the actual corner of the L. This stabilizes the gesture detection, albeit at the cost of a slight displacement.

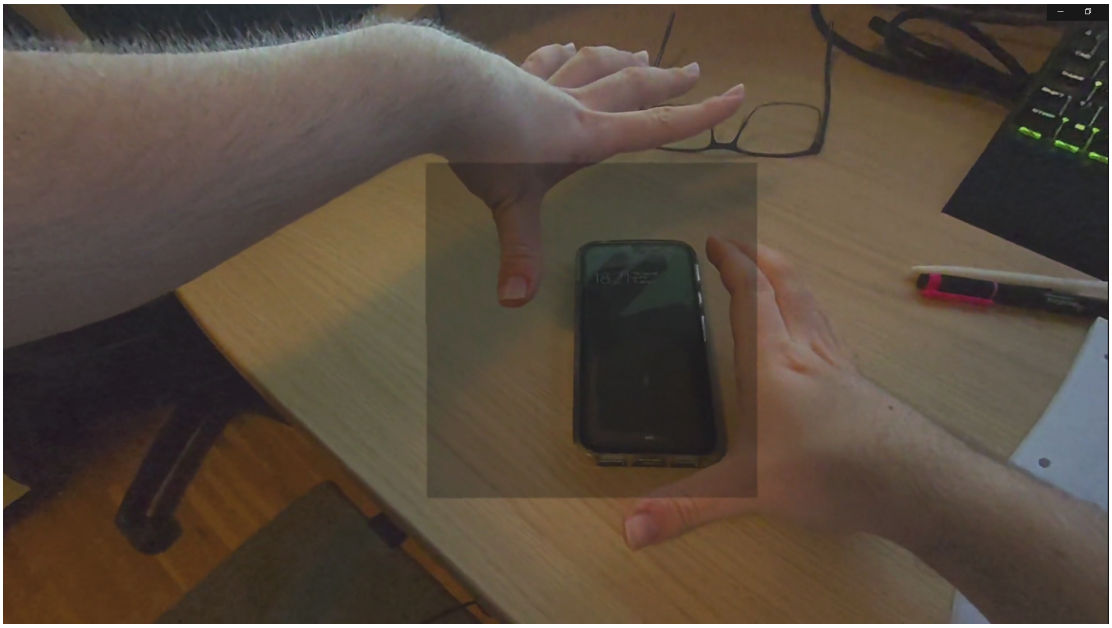


Figure 4.3: Area Selection of the visualized mask

By applying this method to both hands, the selection state can be confirmed. This ends up handing over the position in the user's viewport to the inpainting logic, which then uses it for defining the area to mask of the image, as seen in figure 4.3.

The mask display is managed through a simple RectMask2D from Unity, which is positioned and resized according to the hand positions in the SelectionUiController.

Behind the RectMask2D component, a 2D Canvas can be found. This canvas is

placed according to the camera position during the selection. After the selection, it needs to be rotated to always face the camera, as it is just a 2D display in the 3D Space.
In case the camera moves away from the mask, causing it to be outside of the camera's view frustum, it is frozen in place to avoid any additional unnecessary performance degradation. This is accomplished by comparing the corner points of the mask to the camera's frustum.

Rectangle to Image Pixel-Mapping

For the inpainting algorithms, this mask rectangle needs to be converted into a list of pixels, which need to be inpainted by the algorithms. Since the mask does not change between frames, this only needs to be calculated once. Most algorithms need to regularly check if a pixel is within the masked area. Therefore, a HashMap is used for its fast O(1) access when checking whether it contains a value.

For this conversion, the mask's position needs to be converted into texture coordinates first, which are then used to calculate the array indices for the pixel array returned by the camera texture.

Both the mask and the current frame camera texture are then handed over to the inpainting algorithm for each frame.

4.3 Inpainting Algorithm Implementation

The Inpainting algorithms are being called each frame. The reason for this is to allow for an easy comparison of the performance by looking at the Frames per Second (FPS) the application achieves. The target FPS rate for real-time should be at least 60 FPS for HMDs to avoid motion sickness. For regular displays, 30 FPS should also be sufficient [Wan+23]. The following algorithms were implemented:

- Fast Marching Method (FMM), as defined by Telea [Tel04] and presented in section 2.3.
- Nonlocal Texture Matching and Nonlinear Filtering (NLTM), proposed by Ding, Ram and Rodriguez [DRR19] and explained in section 2.3.
- Exemplar-Based Image Inpainting with modified Priorities (ELTM), first presented by Deng et. al. [DHZ15] and shown in section 2.3.
- Large Mask Inpainting with Fourier Convolutions (LaMa), a pre-trained image inpainting model by Suvorov et. al. [Suv+22] which is explained in section 2.3.

As both the NLTM and ELTM algorithms require downscaling, this too is applied to the FMM and the LaMa methods to keep the input to the algorithms similar. The default resolution of 1280x1280 would mean 100k+ iterations for the texture matching algorithms for the candidate evaluation, which would make both methodologies infeasible for real-time inpainting with current hardware, but by simply downscaling the image and upscaling it again after the inpainting algorithm finishes, the performance can be improved significantly. As the inpainting aims to overlay objects with background predictions, which are not in focus for the user, the quality loss is acceptable and should still be superior in quality compared to FMM due to the texture recognition.

These inpainting implementations all inherit the public Inpaint method from the AbstractInpaintingAlgorithm, see program 4.1, which handles all the evaluation triggers and measurements, whilst the algorithm classes only need to implement the abstract InpaintLogic method. The class for the concrete selected algorithm gets created through a factory method.

Program 4.1: AbstractInpaintingAlgorithm class, which all the inpainting implementation classes inherit from.

```
1 public abstract class AbstractInpaintingAlgorithm {
2     ...
3
4     public Texture2D Inpaint(Texture2D source, HashSet<int> maskPixelIndices) {
5         m_watch.Reset();
6         m_watch.Start();
7
8         // Save original pixels before inpainting for quality evaluation
9         m_originalPixelBuffer = source.GetPixels32();
10        PixelBuffer = source.GetPixels32();
11
12        var texture = InpaintLogic(source, maskPixelIndices);
13
14        m_watch.Stop();
15        // Evaluator calls
16        ...
17
18        return texture;
19    }
20
21    protected abstract Texture2D InpaintLogic(Texture2D source, HashSet<int>
      maskPixelIndices);
22 }
```

Fast Marching Method

One optimization in the algorithms implementation is the min-heap handling in which any stale entries during the inpainting iterations are skipped. This is in regard to the reference pixel distance, which gets updated for neighbor pixels in case the eikonal function result is lower than the prior distance. In the default algorithm, the specific entry inside the heap would need to be looked for and updated accordingly. In this implementation, a new entry is simply added to the heap, as can be seen in algorithm 4.2.

To find out if a heap entry is stale, an id is stored with the heap entry. For each pixel to inpaint, the most current id is stored inside a separate data structure for reference. If this id does not match, the entry is declared stale, reducing the asymptotic runtime of the distance update from O(n) to O(log n)

Algorithm 4.2: Fast Marching Method MinHeap Stale Entry handling

```
1:  function RunFmmWithHeap(heap, flags, distanceMap, pixels)
        Input: heap, the minHeap containing pixels to process
        Input: flags, state of each pixel (Known, Band, Inside)
        Input: distanceMap, distance values for each pixel
        Input: pixels, image pixel array
        Returns Modified pixels array
2:      while heap.Length > 0 do
3:          node ← heap.Pop()
4:          idx ← node.Index
                                                    ▷ Skip stale entries
5:          if node.Generation ≠ generations[idx] then
6:              continue
            ....                    ▷ For each mask/bounds neighbor of the current pixel
7:          if newDist < distanceMap[nbIdx] then
8:              distanceMap[nbIdx] ← newDist
9:              generations[nbIdx] ← generations[nbIdx] + 1
10:             heap.Push(FmmNode(newDist, nbIdx, generations[nbIdx]))
            ....
11:     return pixels
```

Non-local Texture Matching and Nonlinear Filtering

As previously discussed in the Concept chapter (see 3.2.2), the NLTM algorithm by itself is slow.

One of the proposed optimizations was the PatchMatch-oriented approach of looking for subsequent frames candidates within the surroundings of the prior frame's candidate pixel patches. One advantage of this is the possibility of parallelization, as the candidate patches are all independent of one another. Therefore, this can once again be solved through the implementation of an IJobParallelFor as seen in algorithm 4.3

The NLTM Implementation is only configured to use the best 4 candidate patches for the alpha trimmed mean values. This means the found candidates for each target patch need to be sorted in order of texture similarity. As there is a very large number of candidates, of which only the best 4 are required, the overhead of sorting any more candidates is unnecessary.

Therefore, the sort algorithm only uses a partial sort, using the selection sort algorithm, as it is the easiest to implement using deconstruction in C#, as can be seen in algorithm 4.4.

By simply using these optimizations, the Time Per Pixel, which was tracked during the implementation to measure how much time the inpainting algorithm takes in correlation to the pixel count of the mask was reduced by ~10x.

Algorithm 4.3: Nonlocal Texture Matching cached candidate patch search

```
function SearchPositionsJobExecution(SearchPositions, MaskSet, Pixels)
    Input: searchPositions, array of candidate patch positions
    Input: maskSet, hash set containing mask pixel indices
    Input: pixels, image pixel array
    Input: gaussianWeights, weights for texture comparison
    Input: targetX, targetY, width, height, index (for parallelization)
    Returns results array containing candidate indices and distances
    pos ← searchPositions[index]
    sx ← pos mod Width
    sy ← ⌊pos/Width⌋ ... ▷ Only patches in which no particle is within the mask is used
    distance ← CalculateTextureDistance(targetX, targetY, sx, sy, maskSet, pixels,
    gaussianWeights, width, height)
    if distance ≥ 0 then
        results[index] ← (Index: pos, Distance: distance)
    else
        results[index] ← (Index: −1, Distance: ∞)
    return results
```

Algorithm 4.4: Nonlocal Texture Matching candidate Patch Partial Sort

```
function SortCandidates(candidates, K)
    Input: candidates, candidates list with distance
    Input: K, amount of candidates to sort
    Returns candidates list with the first K entries being the entries with the
    smallest distances in the list
    for i ← 0 to K − 1 do
        minIdx ← i
        minDist ← candidates[i].Distance
        for j ← i + 1 to length(candidates) − 1 do
            if candidates[j].Distance < minDist then
                minDist ← candidates[j].Distance
                minIdx ← j
                                          ▷ Swap minimum element to position i
        if minIdx ≠ i then
            swap(candidates[i], candidates[minIdx])
    return candidates
```

Exemplar-Based Image Inpainting with modified priorities

The implementation of this algorithm is, in large parts, similar to the NLTM Algorithm as discussed prior, although there are several large changes:

- The Priority Function was overhauled according to the thesis presented in section 2.3. Due to these changes, the fill order of the masked area stays accurate and

does not degrade. This means the patches targeted for inpainting are of higher quality, and the overall filled area should be more consistent.

- Instead of using an alpha trimmed mean value for the pixels, the best patch is picked. This makes the algorithm a lot more performant, albeit at the cost of artifacts in case the best matching patch contains any noise or artifacts.
- Whilst the NLTM Algorithm used a performance heavy gaussian-weighted distance for its candidate selection, ELTM uses a simple sum of squared differences. This means ELTM will be worse for matching textures, but in return, it should provide superior performance.
- The candidate search is limited to a local area instead of the entire image, although the modified NLTM algorithm uses a similar approach, albeit using the local area of the prior frames candidate patches rather than the target patches local area.

The local search can once again be implemented as a parallel job, allowing for full parallelization in the candidate search. Algorithm 4.5 shows the canidate patch distance calculation of ELTM using the sum of squared differences.

Each patch gets its texture distance calculated accordingly, of which the patch with the lowest distance and therefore the highest similarity is picked and used to fill out the missing pixels of the target patch with the corresponding pixels of the selected source patch.

Large Mask Inpainting Model Integration

As training a new inpainting model lies out of scope for this thesis, the provided model[8], which was trained by the original LaMa authors back in 2021 for their publication (see [Suv+22]), was used.

The Unity developers created the Sentis library[9] for the integration of pretrained neural network models into applications for real-time use. This library also allows for those models to be run directly on the GPU [Tec26b][10].

This library only offers support for models in the Open Neural Network Exchange (ONNX)[11] format, whilst the provided LaMa model is merely a Checkpoint file. Therefore, this model needs to be converted accordingly. This has already been implemented by Carve-Photos, who provided a Python script[12] to export the model to ONNX.

By using this script and exporting the model with the Opset-Version 15, it can then be imported by the Sentis engine during the application's runtime, as seen in program 4.2.

Both the source image and the inpainting mask are both put into the model through tensors. The size of these needs to be defined during the ONNX export and has been set to 256x256 pixels for the evaluation. Therefore, the input texture and mask need to

[8] https://github.com/advimman/lama

[9] https://docs.unity3d.com/Packages/com.unity.ai.inference@2.5/manual/index.html

[10] There have been attempts to get the model to run on the GPU, but it caused several crashes due to running out of GPU memory whenever a screen recording is running in parallel. In addition to this, the entire OS would become unresponsive, which made the GPU computation of this model unusable. Therefore, it was evaluated using the CPU, just like the other approaches.

[11] https://onnx.ai/

[12] https://github.com/Carve-Photos/lama/tree/main

Algorithm 4.5: Exemplar-Based Image Inpainting local candidate search difference calculation

```
function CalculateSSD(targetX, targetY, sourceX, sourceY)
    Input: targetX, targetY, sourceX, sourceY: patch center pixels of source and target
    Input: pixels, array containing the image pixels
    Input: maskSet, hash set containing the pixel indices of the mask pixels
    Input: width, height of the image
    Returns candidates list with the first K entries being the entries with the smallest distances in the list
    Input: patch centers and image data
    Output: normalized SSD distance
    sum ← 0
    cnt ← 0
    for dy ← −r to r do
        for dx ← −r to r do
                                        ▷ Compute pixel indices (boundary checks omitted)
            targetIdx ← (targetY + dy) · width + (targetX + dx)
            sourceIdx ← (sourceY + dy) · width + (sourceX + dx)
            t ← pixels[targetIdx]
            s ← pixels[sourceIdx]
            d_r ← t_r − s_r
            d_g ← t_g − s_g
            d_b ← t_b − s_b
            sum ← sum + d_r^2 + d_g^2 + d_b^2
            cnt ← cnt + 1
    if cnt > 0 then
        return sum/cnt
    else
        return ∞
```

both be downscaled accordingly and upscaled afterwards, similarly to the NLTM and ELTM implementations.

Running the model is just as simple, with just a few lines of code, as seen in program 4.3, to create the tensors and schedule the model run. The result can then be retrieved by the PeekOutput method, which blocks the thread until the model finishes.

This output tensor then gets converted back into an array of color values, which is then used to create a new texture for Unity to handle.

In terms of implementation complexity, it was by far the easiest to integrate, albeit due to all the complexity lying inside the training data of the model.

4.4 Limitations

As this implementation is meant as a prototype and the Meta Quest Passthrough API is still new, there are several limitations regarding the implementation:

Program 4.2: LaMa Model Initialization

```
1 private void Initialize() {
2     ...
3
4     // Load the model asset from Resources folder
5     var modelAsset = Resources.Load<ModelAsset>(ModelResourcePath);
6
7     // Load the model from the asset
8     m_model = ModelLoader.Load(modelAsset);
9
10    // Create worker with CPU backend for Quest, for GPU BackendType.GPUCompute
11    m_worker = new Worker(m_model, BackendType.CPU);
12
13    ...
14 }
```

Program 4.3: LaMa Model Inference

```
1 private Tensor<float> RunInference() {
2     // Tensors become GPU−bound after inference and can't be rewritten
3     var imageShape = new TensorShape(1, 3, ModelSize, ModelSize);
4     var maskShape = new TensorShape(1, 1, ModelSize, ModelSize);
5     using var imageTensor = new Tensor<float>(imageShape, m_imageData);
6     using var maskTensor = new Tensor<float>(maskShape, m_maskData);
7
8     m_worker.SetInput("image", imageTensor);
9     m_worker.SetInput("mask", maskTensor);
10
11    m_worker.Schedule();
12
13    var tensor = m_worker.PeekOutput() as Tensor<float>;
14
15    // Download from Tensor to CPU − ReadbackAndClone returns a new CPU−readable tensor
16    var cpuTensor = tensor.ReadbackAndClone();
17    return cpuTensor;
18 }
```

- The Meta Quest Passthrough API, as of this moment, only supports fixed, small camera feed textures, which do not contain all the content visible in the AR application background, which is applied to the sky box. This means the inpainting area is limited around the middle of the user's viewport and cannot reach the edges yet.
- In addition to the smaller size of the camera textures, they are applied on a 2D plane in the 3D space. Due to the AR background being applied to the skybox, it means the distortion, which is usually not visible on the skybox causes the overlay of the inpainted texture to be slightly offset in some cases. Whilst this could be avoided by calculating the correction transformation and applying it, this would be out of the scope of this thesis, as it is not too visible on the inpainted areas

as long as the mask leaves some padding between the edges and the object to be removed

- The implementations of the inpainting algorithms and evaluations are built for the CPU, due to the large complexity and, in part, major rewrites which are necessary to get the algorithms and data structures to run on GLSL shader code instead of C#.
- The mask does not adjust to the objects behind it. If a hand is placed behind the mask or the object behind the mask is moved, the mask will not adjust for the change, painting over the hand or the now-empty space. This could be solved by adjusting the mask depending on the objects behind it, but this would once again add more performance degradations and complexity due to needing to track the object's movements and reacting to changes in the environment.

Chapter 5

Evaluation

The main objective of this thesis was to investigate which inpainting algorithms are suited for a real-time visual clutter reduction in AR. In this chapter, different scenarios are evaluated for each algorithm in terms of runtime performance, overall inpainting quality and the visual clutter reduction.

The entire testing was conducted on a Meta Quest 3 running on the current release of HorizonOS, v81. As the metrics are calculated as average values, the algorithm is allowed to run until the value stabilizes to avoid any outliers to invalidate the data.

For the evaluation, four test scenarios have been prepared, which are aimed at bringing out both the conceptual advantages and disadvantages of the different implemented methodologies:

- The first test case serves as a baseline, by having the surroundings in a uniform color. For this, a simple white table with a white wall in the background is used. On the table there is a variety of clutter to hide.
- To increase the complexity and validate the pattern and texture synthesis, the second test case is done on a wooden table. In terms of quality, the evaluated approaches should recreate the wood pattern in a believable manner.
- As the clutter may be spread out in some cases, it is important to evaluate the performance and quality loss for large areas. Therefore the third test scenario aims to hide an entire keyboard taking up most of the cameras input. This serves to challenge the algorithms, both in terms of the available image area to use for source pixels, as well as how well patterns and information are retained when moving towards the masks center.
- The last test case aims to test how well the evaluated methodologies handle reflective objects, as they may be picked up as plausible source pixels, even though they are distorted by the glass.

These scenarios are evaluated on the following metrics:

- For performance the framerate is measured. This framerate is measured both in total and isolated for the inpainting algorithm. This helps to separate the overhead caused by the camera texture segmentation and mask preparation from the actual inpainting compute time.
- The quality is evaluated by using the metric presented in chapter 3.3. This Image

Inpainting Quality Assessment (IIQA) considers both the naturalness and the gradients at the masks edge to calculate an estimation of the inpainting quality.

- As the reason for overlaying inpainted content on the image is to reduce clutter, the clutter reduction should also be measured. To accomplish this, the clutter measurement algorithm of Rosenholtz, Li and Nakano as presented in chapter 3.3 is used.

5.1 Evaluation Background Jobs

Since the quality and clutter evaluation algorithms need a lot of computational resources, they cannot run in the foreground for each frame, due to it affecting the performance evaluation severely. Therefore, the Unity Job System is used for these evaluations, albeit with the jobs running in the background rather than blocking the main thread like in the inpainting implementations.

This means the quality and clutter evaluation works on a "evaluate if ready" principle, only evaluating frames whenever there is no other frame being evaluated at the moment of the call, as seen on figure 5.1. Whenever no job is running the data is prepared for the evaluation job by moving them into more efficient data structures for the evaluation algorithms. In case a job is finished between two evaluation scheduling requests, the result of the job is saved before scheduling the next evaluation job. The result cannot be saved into the result data structures from the job itself as the Burst Compiler running the background job does not allow for the usage of C# containers and other complex data structures.

Whilst this reduces the consistency of the evaluation results slightly, it solves the issue of the evaluation impacting the performance of the rest of the program, as it is evaluated in a background thread.

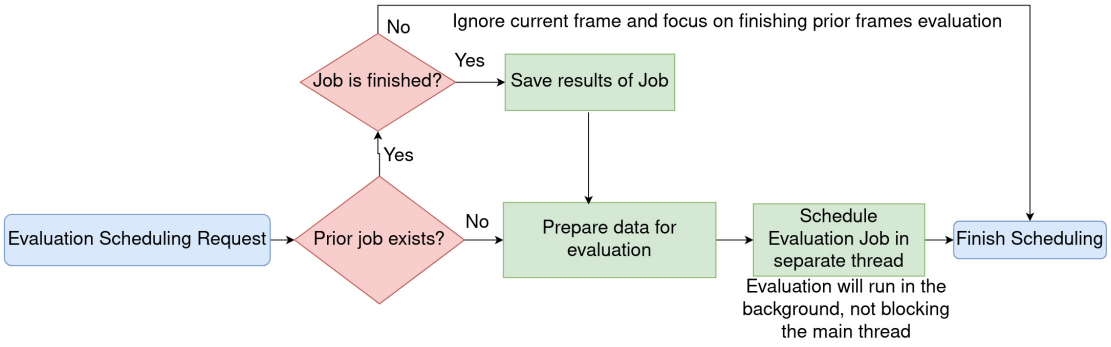


Figure 5.1: Flowchart of the background evaluation job scheduling

Performance, on the other hand, is easy to calculate and aggregate. Therefore, it can be calculated each frame.

5.2 Uniform Background

A uniform background serves as a baseline testcase in which the surroundings should be plain, in the best case, one uniform color. The original baseline image used for the evaluation can be seen in figure 5.2. Since the surrounding background does not contain any patterns or textures, it should be reasonably easy for any inpainting method to achieve a high-quality result. At most, there should be some artifacts in terms of the shading or through the surrounding objects.

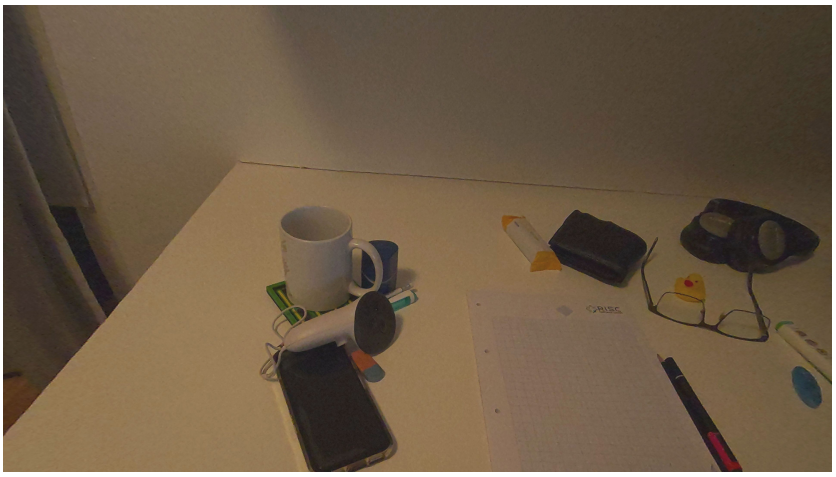


Figure 5.2: Uniform background test scenario baseline image containing multiple distracting objects on a white table

The results in figure 5.3 show good handling of uniform backgrounds from each of the algorithms. FMM shows slight visual artifacts in the middle of the mask. This can be correlated to the band filling principle of the algorithm. Since it looks like there is only one pixel missing, it can be assumed no band could be created any more and it could not inpaint. This would require additional handling, which would cost additional performance per pixel. Therefore this small trade off was accepted, as it only rarely happens. Both ELTM and LaMa produced very clean results overall, except for the shading not matching up with the background. This cannot be avoided with the given algorithms, but should not prove too distracting either as the inpainting is meant to be in the background. NLTM seems to have attempted to recreate the shading, as seem with the darker line in the bottom middle of the mask, although the shading still does not fit.

In terms of performance, as seen in figure 5.4, FMM performed the best, as expected, followed by ELTM. Surprisingly enough, even though the LaMa model is optimized for performance, it performed by far the worst, even compared to NLTM. As prior tests with the GPUCompute have shown a similar performance, the most likely culprit is the inference time for the model, as if it were bottlenecked by the CPU, the prototyping with GPUCompute should have led to significant runtime improvements.

In terms of quality, according to the IIQA, the other algorithms performed significantly worse than the LaMa model. Surprisingly enough, whilst all algorithms were

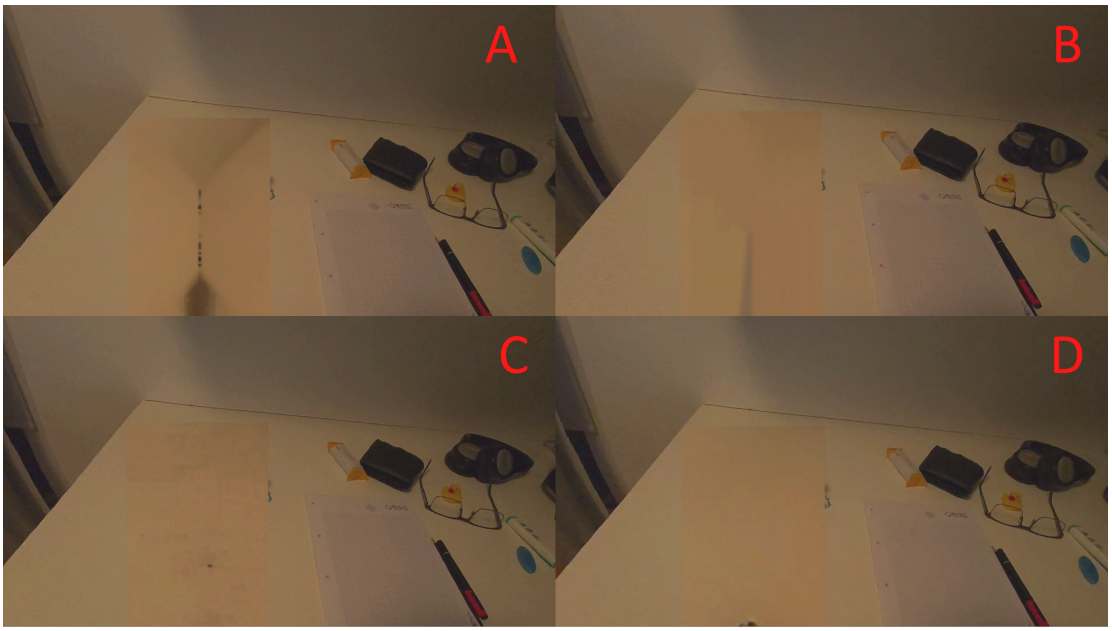


Figure 5.3: Uniform background test scenario result image comparison for each algorithm (A = FMM, B = NLTM, C = ELTM, D = LaMa)

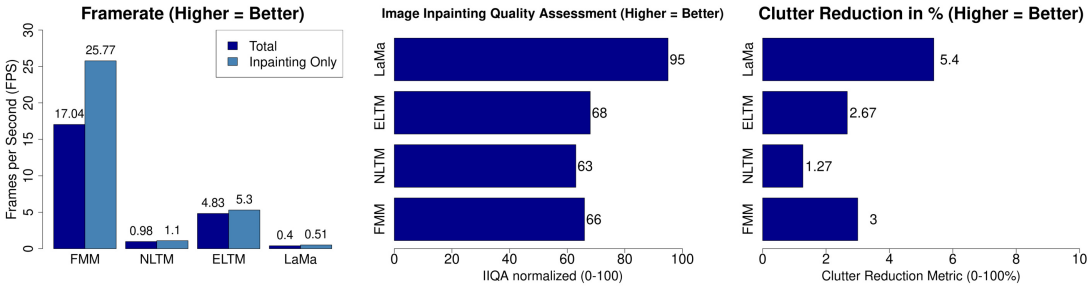


Figure 5.4: Uniform background test scenario: Evaluation result charts for performance and quality

given an image with the same downscale quality, LaMa appears to create fewer artifacts, as one can see in figure 5.3. This is also correlated in the clutter reduction metric, as the noise created by the other methodologies was picked up as additional visual clutter by the visual clutter algorithm, compared to the cleaner result of LaMa. Another observation in terms of quality is the superior quality of FMM in comparison to NLTM and ELTM. This is most probably once again correlated to the visual noise. The concept of FMM, in which surrounding pixels are propagated inwards of the mask, seems to cause a lot fewer visual artifacts during upscaling compared to NLTM and ELTM, which causes the end result to look less grainy.

5.3 Textured Background

The environment of this test scenario is a wooden desk, as the texture of the wood provides a good example of how well the inpainted area can blend in with textured backgrounds, as seen in figure 5.5.

Figure 5.5: Textured background test scenario baseline image containing several distracting objects on a wooden table

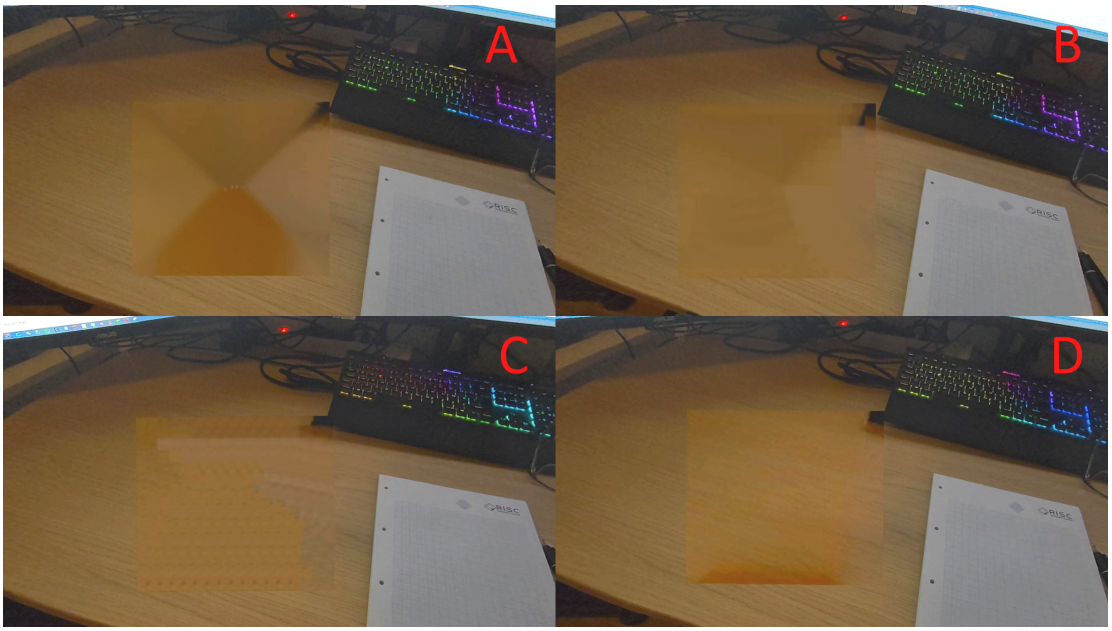


Figure 5.6: Textured background test scenario result images for each of the algorithms (A = FMM, B = NLTM, C = ELTM, D = LaMa)

When looking at the results shown in figure 5.6, one can immediately notice the stark contrast in quality between the algorithms. Whilst FMM does not keep any of the pattern detailing, the other algorithms seem to do a reasonably good job. NLTM and ELTM both seem to struggle a bit due to the downscaled input image, although both seem to be able to keep the texture reasonably well. Due to the reuse of the patches in ELTM, it simulates the wood texture a lot better, getting rid of the blurry artifacts of NLTM. LaMa seems the least affected by the low resolution input image, presenting a detailed wood pattern, albeit with a small offset due to the alignment issues, which were mentioned in the limitations chapter 4.4.

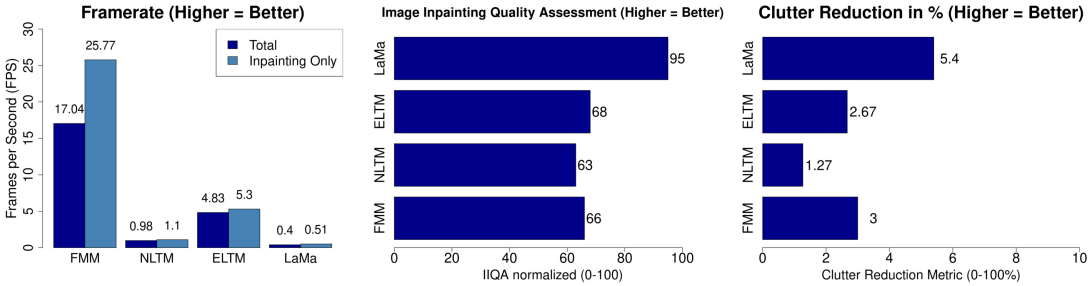


Figure 5.7: Textured background scenario quality and performance evaluation result barcharts

The performance metrics of this test case, as can be seen in figure 5.7, are very similar to the metrics of section 5.2, albeit with the framerate being lower overall. This can be correlated to the larger mask which needs to be inpainted in this test scenario.

In terms of the quality metrics, the visual analysis is being confirmed. ELTM performed very well in comparison to NLTM and FMM, which both ended up with a worse quality assessment compared to the prior test scenario. The clutter reduction, too, ended up completely different, with NLTM and FMM being about on par with each other. Most probably due to not being able to reproduce the background pattern properly and in a high enough quality. NLTM is most probably suffering here from the missing source information due to the downscaled source image.

ELTM seems to handle the downscaled source image very well though, still delivering a result, which reduces the clutter reasonably well whilst producing a texture which can blend in well. But it is still significantly overtaken by the LaMa model, which pays for its quality with the slow runtime. It lags behind the other approaches by far as clearly illustrated by the metrics in figure 5.7.

Compared to the last scenario, whilst the performance has decreased slightly due to the larger mask, the relative differences of the algorithms stayed the same. In terms of quality and clutter reduction, FMM achieves worse results in relation to the other algorithms, when compared with the uniform background scenario, due to it being unable to imitate patterns, whilst ELTM shows its strengths of being able to generate continuous patterns reasonably well, outperforming NLTM and FMM marginally in comparison to the uniform background scenario, in which FMM was still above. For LaMa, all the metrics stayed similar compared to the prior scenario.

5.4 Large Area Inpainting

As the aforementioned test scenarios are focused on small, targeted inpainting masks, this test case aims to test the performance and quality for larger masks, mostly in regard to stress-testing the implementations in a difficult but plausible scenario. For this, the keyboard shown in figure 5.8 should be replaced by the wood texture as even as possible. Such a use-case might be interesting when trying to hide physical objects before placing smaller virtual objects over them to avoid, in this case the keyboard, to distract from the virtual object in front of it.

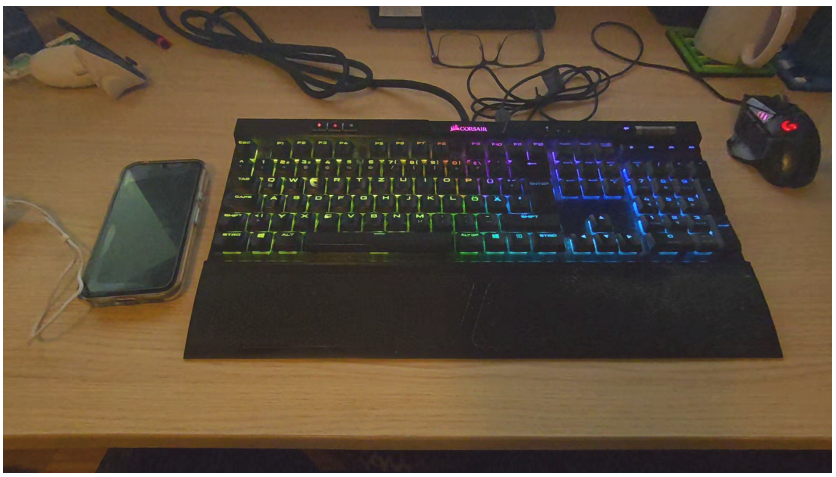


Figure 5.8: Large area inpainting background test scenario baseline image containing a centered keyboard, which would block and distract from overlaid virtual objects

Interestingly enough, when looking at the comparison, FMM and NLTM look almost similar. The only way to differentiate between the two is the slight graininess and what looks like the wood pattern in the bottom right part of the overlay. This scenario defines a large part of the viewport as the mask, therefore, the areas available to use for the candidate search are very limited. It seems like the quality of the NLTM inpainting suffered from it, whilst the ELTM inpainting seems to have benefited from it.

The ELTM algorithm performed surprisingly well in this scenario. Whilst the performance degradation is definitely noticeable, both the quality and the clutter reduction are high. Considering one of the difficulties with large masks is the loss of structural information and patterns, the algorithm was able to keep a consistent pattern very well. The other algorithms performed as expected, considering the last test cases, although NLTM losing more performance compared to LaMa might seem questionable at first, it is to be expected, as LaMa was optimized for such large masks.

5.5 Reflective Surroundings

The focus here is to test how well the inpainting algorithms can consider reflective objects in both the inpainting mask, as well as its surroundings for the candidate selection

Figure 5.9: Large mask test scenario result images for each of the analyzed algorithms (A = FMM, B = NLTM, C = ELTM, D = LaMa)

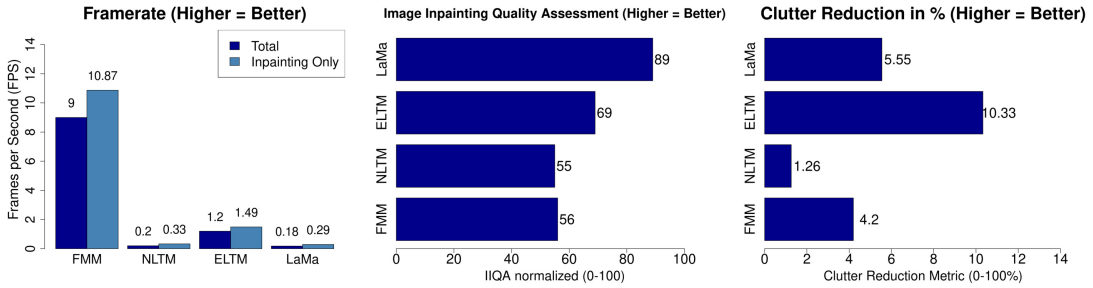


Figure 5.10: Large mask test scenario evaluation result charts in terms of both performance and quality

of NLTM and ELTM. These objects should preferably be avoided as sources due to the discoloring and blurring of the reflected background in favor of the regular background.

Based on the inpainting concepts, the initial assumption was to only see quality degradations for NLTM and ELTM, which were confirmed, as seen in figure 5.12. Surprisingly enough, the results of the LaMa model became a lot more blurry as well. This is most likely because of the slight blurryness caused by the glasses in the surroundings influencing the quality of the generated overlay. Whilst FMM may at first look like it also degraded, the black patch on top is a normal behavior of FMM, as it picked up the pixels of the keyboard above as its reference pixels, as there werent any others in the proximity.

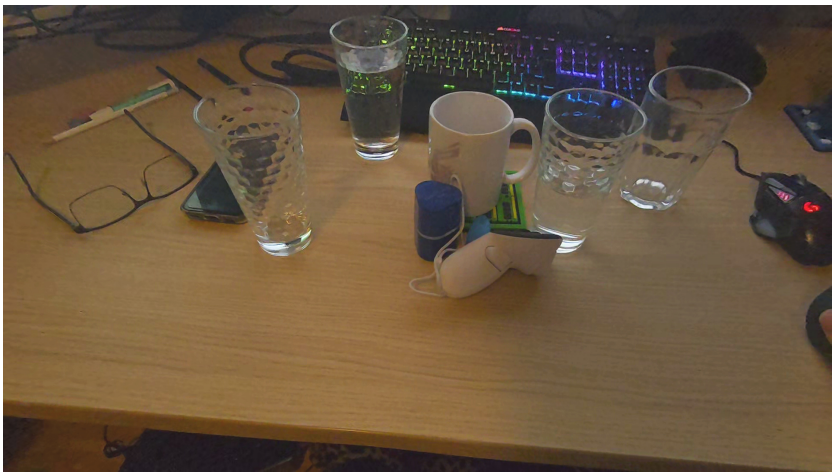


Figure 5.11: Reflective surroundings inpainting background test scenario baseline image containing clutter surrounded by multiple partially filled and empty glasses of water

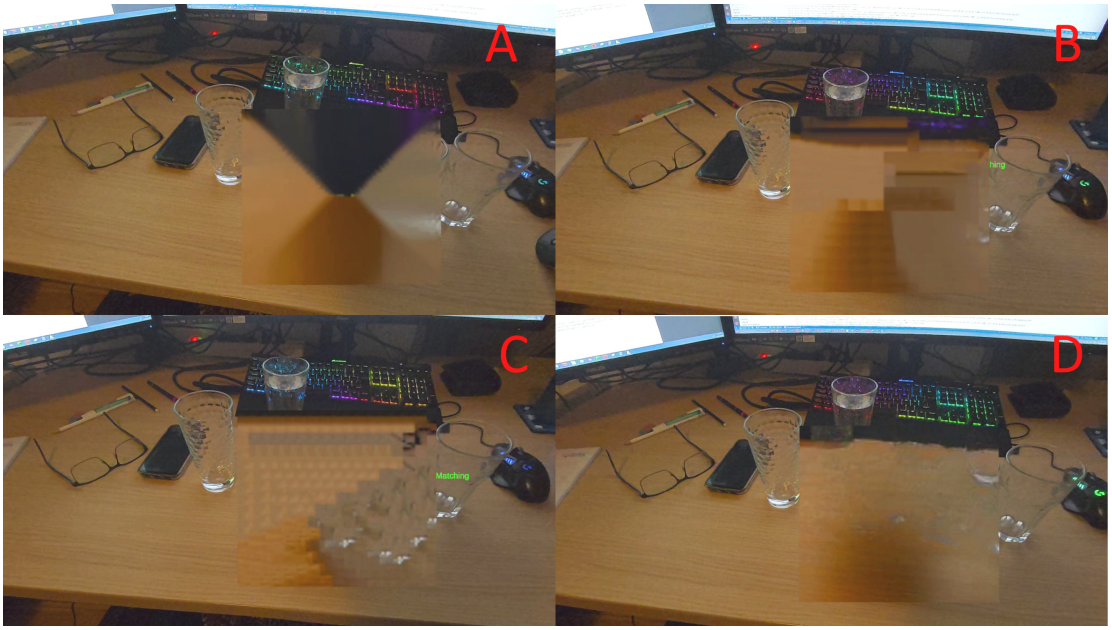


Figure 5.12: Reflection surroundings test scenario result images for each of the analyzed algorithms (A = FMM, B = NLTM, C = ELTM, D = LaMa)

In addition ELTM seems to have kept the pattern of the glassware, taking the color of the wood but keeping the gradients of the glass, which is quite an interesting behavior.

In terms of performance, the result was once again as expected, but performance was not the focus of this test case.

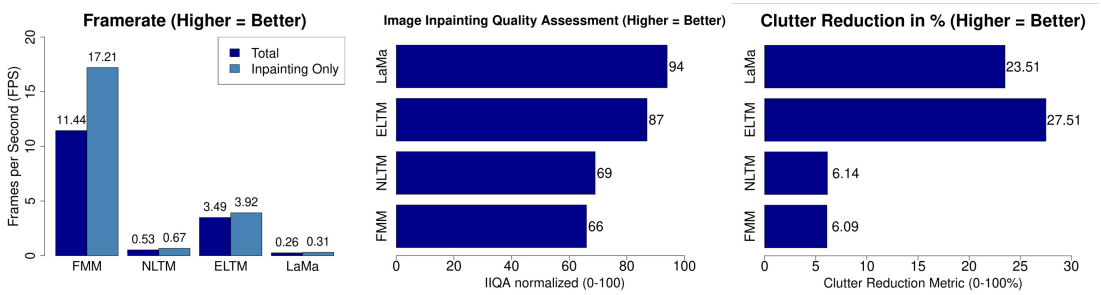


Figure 5.13: Reflection surroundings test scenario evaluation result charts for performance and quality

Considering the quality, it seems the IIQA and the clutter reduction algorithms both are not suited well for such use cases, as the values for ELTM and LaMa are high, even though the resulting images do not look natural. This may be in regard to how the boundaries are considered for quality and clutter removal, whereas the pattern of the glass may have unintentionally become the reference pattern for the quality evaluation.

Evaluation Summary

Whilst FMM performed outstandingly in terms of performance, outperforming the other algorithms by up to 10x of their framerate, it suffered severely in terms of quality outside of a uniform background. NLTM was supposed to provide the best quality, albeit at the cost of performance. Due to the downscaling of the input image to make the algorithm usable, the evaluation showed NLTM losing its advantage, producing mediocre results at best, as shown in the quality metrics, which are the lowest in every scenario except the reflective surroundings.

Both ELTM and LaMa look very promising in terms of quality though, providing high results with scores of 68 to 87 for ELTM and 89 to 94 in the IIQA and outperforming both FMM and NLTM in terms of clutter reduction in every scenario, albeit at the cost of performance in comparison to FMM. ELTM might still be manageable for close to real time inpainting, with most scenarios hovering around 1.5 to 5 FPS, but LaMa might not be feasable as of this moment with its lackluster performance. This may change in the future though, whenever the inference time of neural networks decreases further.

In general, the quality of the inpainting algorithms look really promising and the metrics suggest them being helpful for reducing clutter. None of the algorithms managed to achieve real-time framerate results whilst running on the CPU though. This may possibly be alleviated by migrating the algorithms to GPU compatible code or by changing the system to not require the inpainting to occur every frame, giving up reactivity in the inpainting for a more consistent system performance.

Chapter 6

Conclusion and Future Work

Visual clutter is a severe problem for AR applications due to information overload and the distraction it causes. Through the use of inpainting and DR, it is possible to hide this clutter from the user.

To accomplish this, it is important to choose a fitting inpainting method, which provides high performance at an acceptable quality level. To provide such information, several approaches were analyzed in detail and implemented for evaluation through different evaluation and assessment criteria in terms of both performance and quality. Whilst the approach of the developed prototype, which evaluates the inpainting algorithm every frame, is not practical and should probably be replaced by an event-driven approach for proper use, it serves as a good evaluation of the approach's performance. In terms of evaluation criteria, for quality, the IIQA, which was proposed by Liang et. al. [LIA+19] was implemented.

As NLTM at first seemed unsuited for real-time inpainting, a combination of the PatchMatch algorithm, proposed by Barnes et. al. in 2009[Bar+09], with the existing NLTM algorithm led to major performance improvements, although as later proven in chapter 5, the performance improvements were still nowhere near enough to achieve real-time performance.

In general, none of the methods really reached the performance necessary for real-time inpainting, but for cases like the clutter removal in the background, such performance might not be necessary, in case slight delays in reactivity and feedback of the inpainting can be accepted.

In the evaluation, it was noticed that whilst FMM provides the best performance in every scenario by several magnitudes, the ELTM approach provides much higher quality results in many cases, albeit at the cost of performance. Both NLTM and LaMa, as of this moment, should not be considered at all. Whilst LaMa seemed to scale well with larger masks, it seems the initial overhead of the inference and initialization inside the model makes real-time usage infeasible.

6.1 Future Work

Whilst this thesis serves as a good starting point, it is possible to branch off in many different directions for further exploration of the usages for real-time inpainting and

clutter removal. The approach for area selection, as presented in chapter 3, is easy to use and serves its purpose. However, there are several advancements to automatically detect clutter based on a users' selection. In 2023, an article by Huynh, Zhou et. al. [Huy+23] proposed a methodology to easily detect similar object patterns within an image. This may be used to allow the user to simply select an object by pointing at it, followed by the SimpSON algorithm creating masks around the image for the necessary inpainting.

In addition to this, it might be interesting to conduct a user study on the effects of hiding distractions using inpainting in AR, how it affects the user and if there are any differences to other methods of hiding objects. Whilst the work of Cheng et. al [Che+22] has already laid good groundwork on the effects of DR, inpainting was not considered at that time and may outperform the approaches which were analyzed.

The proposed and implemented evaluation prototype was built in an easily extendable manner, it would be interesting to evaluate more methodologies and algorithms, as there are still many promising algorithms which could not be analyzed in further detail, not to mention newer algorithms and methodologies which might be published over the next few years.

As this thesis was heavily focused on CPU-based methodologies, converting these algorithms into GPU-compatible versions and comparing these should definitely be addressed in the future to ensure the best performance of the algorithms.

Appendix A

Source Code

The entire source code can be accessed at https://github.com/Shebbyy/InpaintAR. It also contains all the LaTeX files and resources created for this thesis.

Unity Burst Compiler

Since the Mono runtime of Unity is rather slow for such high-performance implementations, the Unity3D Engine provides the Burst Compiler[1].

This compiler works by translating the Intermediate Language code generated by C# into code specifically generated for the target CPU architecture by using LLVM[2], which can lead to a 10x - 100x performance improvement, similar to writing raw C/C++ code. Since the LLVM translation layer does not support the full C# language scope, Burst code needs to be written in a specific pattern and marked with the [BurstCompile] Attribute.

In terms of the code written, it can be seen as similar to C in its approach. It only supports structs, needs manual memory management and only offers very simple data structures. It is commonly used in the Unity Job system and requires a similar approach for calling it in a method, although it is also possible to compile static methods using the burst compiler [Sch24].

In the FMM inpainting implementation, the job for the initialization is called, as seen in A.1.

[1] https://docs.unity3d.com/Packages/com.unity.burst@1.8/manual/index.html
[2] https://llvm.org/

Program A.1: Fast Marching Method Initialization Job call structure

```
1 private void InpaintFmmBurst(HashSet<int> maskPixelIndices) {
2     // normal C# Container cannot be used, Unity provides NativeContainer Structs as
      replacements
3     var distanceMap = new NativeArray<float>(m_pixelCount, Allocator.TempJob);
4     ...
5     var initJob = new InitializeFmmJob {
6         DistanceMap = distanceMap,
7         ...
8     };
9     initJob.Schedule(m_pixelCount, 64).Complete(); // schedule for each pixel with 64 in
      parallel
10    ...
11
12    // manual disposal of all instantiated data structures
13    distanceMap.Dispose();
14 }
15
16 [BurstCompile]
17 private struct InitializeFmmJob : IJobParallelFor {
18    [WriteOnly] public NativeArray<float> DistanceMap;
19    ...
20    public void Execute(int index) {
21        ...
22    }
23 }
```

List of Figures

List of Tables

References

Literature

[Bar+09] Connelly Barnes et al. "PatchMatch: a randomized correspondence algorithm for structural image editing". *ACM transactions on graphics* 28.3 (2009), pp. 1–11 (cit. on pp. 17, 43).

[Ber+00] Marcelo Bertalmio et al. "Image inpainting". In: *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '00. USA: ACM Press/Addison-Wesley Publishing Co., 2000, pp. 417–424. DOI: 10.1145/344779.344972 (cit. on p. 8).

[Che+22] Yi Fei Cheng et al. "Towards Understanding Diminished Reality". In: *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. New York, NY, USA: ACM, 2022, pp. 1–16 (cit. on pp. 1, 2, 5, 6, 44).

[CJL23] Muzi Cui, Hao Jiang, and Chaozhuo Li. "Progressive-Augmented-Based DeepFill for High-Resolution Image Inpainting". *Information (Basel)* 14.9 (2023), p. 512 (cit. on p. 2).

[CJM20] Lu Chi, Borui Jiang, and Yadong Mu. "Fast Fourier Convolution". In: *Neural Information Processing Systems*. 2020. URL: https://api.semanticscholar.org/CorpusID:227276693 (cit. on p. 12).

[DHZ15] Liang-Jian Deng, Ting-Zhu Huang, and Xi-Le Zhao. "Exemplar-Based Image Inpainting Using a Modified Priority Definition". *PloS one* 10.10 (2015), e0141199 (cit. on pp. 11–13, 25).

[DRR19] Ding Ding, Sundaresh Ram, and Jeffrey J. Rodriguez. "Image Inpainting Using Nonlocal Texture Matching and Nonlinear Filtering". *IEEE transactions on image processing* 28.4 (2019), pp. 1705–1719 (cit. on pp. 2, 9, 11–13, 25).

[EL99] A.A. Efros and T.K. Leung. "Texture synthesis by non-parametric sampling". In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*. Vol. 2. 1999, 1033–1038 vol.2. DOI: 10.1109/ICCV.1999.790383 (cit. on p. 1).

[Elh+20] Omar Elharrouss et al. "Image Inpainting: A Review". *Neural processing letters* 51.2 (2020), pp. 2007–2028 (cit. on p. 6).

[GG+03] Raphael Grasset, J-D Gascuel, et al. “Interactive mediated reality”. In: *The Second IEEE and ACM International Symposium on Mixed and Augmented Reality, 2003. Proceedings.* IEEE. 2003, pp. 302–303 (cit. on p. 5).

[Gil06] Alan Gillette. *Image inpainting using a modified Cahn-Hilliard equation.* Vol. 68. 03. 2006 (cit. on p. 6).

[Gon+14] Yuanhao Gong et al. “Image enhancement by gradient distribution specification”. *Computer Vision – ACCV 2014 Workshop* (Jan. 2014), pp. 47–62 (cit. on p. 18).

[Gsa+24] Christina Gsaxner et al. “DeepDR: Deep Structure-Aware RGB-D Inpainting for Diminished Reality”. In: *Proceedings (International Conference on 3D Vision. Online) 3DV*. IEEE, 2024, pp. 750–760 (cit. on p. 2).

[HB10] J Herling and W Broll. “Advanced self-contained object removal for realizing real-time Diminished Reality in unconstrained environments”. In: *2010 9th IEEE International Symposium on Mixed and Augmented Reality*. IEEE, 2010, pp. 207–212 (cit. on p. 14).

[Hon+24] Junlei Hong et al. “Visual Noise Cancellation: Exploring Visual Discomfort and Opportunities for Vision Augmentations”. *ACM transactions on computer-human interaction* 31.2 (2024), pp. 1–26 (cit. on p. 2).

[Huy+23] Chuong Huynh et al. “SimpSON: Simplifying Photo Cleanup with Single-Click Distracting Object Segmentation Network”. In: *Proceedings (IEEE Computer Society Conference on Computer Vision and Pattern Recognition. Online)*. IEEE, 2023, pp. 14518–14527 (cit. on p. 44).

[KL51] S. Kullback and R. A. Leibler. “On Information and Sufficiency”. *The Annals of Mathematical Statistics* 22.1 (1951), pp. 79–86. DOI: 10.1214/aoms/1177729694 (cit. on p. 17).

[KSY16] Norihiko Kawai, Tomokazu Sato, and Naokazu Yokoya. “Diminished Reality Based on Image Inpainting Considering Background Geometry”. eng. *IEEE transactions on visualization and computer graphics* 22.3 (2016), pp. 1236–1247 (cit. on p. 2).

[LIA+19] Song LIANG et al. “Quality Index for Benchmarking Image Inpainting Algorithms with Guided Regional Statistics”. *IEICE Transactions on Information and Systems* E102.D (July 2019), pp. 1430–1433. DOI: 10.1587/transinf.2018EDL8206 (cit. on pp. 17, 18, 43).

[Liu+25] Weimin Liu et al. “Real-time tracking and inpainting network with joint learning iterative modules for AR-based DALK surgical navigation”. *Computer methods and programs in biomedicine* 272 (2025), p. 109068 (cit. on p. 6).

[MA23] Xiaojin Ma and Richard A. Abrams. “Visual Distraction’s “Silver Lining”: Distractor Suppression Boosts Attention to Competing Stimuli”. *Psychological science* 34.12 (2023), pp. 1336–1349 (cit. on p. 2).

[Man02] Steve Mann. “Mediated reality with implementations for everyday life”. *Presence Connect* 1 (2002), p. 2002 (cit. on p. 5).

[MF02] S. Mann and J. Fung. “EyeTap Devices for Augmented, Deliberately Diminished, or Otherwise Altered Visual Perception of Rigid Planar Patches of Real-World Scenes”. *Presence* 11 (Apr. 2002), pp. 158–175. DOI: 10.1162/1054746021470603 (cit. on pp. 1, 4, 5).

[Mil+95] Paul Milgram et al. “Augmented reality: a class of displays on the reality-virtuality continuum”. In: *Telemanipulator and Telepresence Technologies*. Ed. by Hari Das. Vol. 2351. International Society for Optics and Photonics. SPIE, 1995, pp. 282–292. DOI: 10.1117/12.197321 (cit. on p. 1).

[MK94] Paul Milgram and Fumio Kishino. “A Taxonomy of Mixed Reality Visual Displays”. *IEICE Trans. Information Systems* vol. E77-D, no. 12 (Dec. 1994), pp. 1321–1329 (cit. on p. 4).

[Pie+97] Jeffrey S. Pierce et al. “Image plane interaction techniques in 3D immersive environments”. In: *Proceedings of the 1997 symposium on Interactive 3D graphics*. New York, NY, USA: ACM, 1997, 39–ff. (Cit. on p. 15).

[Poe+12] Ronald Poelman et al. “As if being there: mediated reality for crime scene investigation”. In: *Proceedings of the ACM 2012 Conference on Computer Supported Cooperative Work*. CSCW ’12. Seattle, Washington, USA: Association for Computing Machinery, 2012, pp. 1267–1276. DOI: 10.1145/2145204.2145394 (cit. on p. 5).

[Ram+16] Francois Rameau et al. “A Real-Time Augmented Reality System to See-Through Cars”. *IEEE transactions on visualization and computer graphics* 22.11 (2016), pp. 2395–2404 (cit. on pp. 2, 5).

[RLN07] Ruth Rosenholtz, Yuanzhen Li, and Lisa Nakano. “Measuring visual clutter”. *Journal of vision (Charlottesville, Va.)* 7.2 (2007), p. 17 (cit. on pp. 1, 4, 5, 18).

[Set99] J. A. Sethian. “Fast Marching Methods”. *SIAM Review* 41.2 (1999), pp. 199–235. DOI: 10.1137/S0036144598347059. eprint: https://doi.org/10.1137/S0036144598347059 (cit. on p. 7).

[SHD21] Irawati Nurmala Sari, Emiko Horikawa, and Weiwei Du. “Interactive Image Inpainting of Large-Scale Missing Region”. *IEEE Access* 9 (2021), pp. 56430–56442 (cit. on p. 9).

[Sil17] Sanni Siltanen. “Diminished reality for augmented reality interior design”. *The Visual computer* 33.2 (2017), pp. 193–208 (cit. on p. 5).

[Suv+22] Roman Suvorov et al. “Resolution-robust Large Mask Inpainting with Fourier Convolutions”. eng. In: *2022 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2022, pp. 3172–3182 (cit. on pp. 12, 13, 25, 29).

[Tel04] Alexandru Telea. “An Image Inpainting Technique Based on the Fast Marching Method”. *Journal of Graphics Tools* 9 (Jan. 2004). DOI: 10.1080/10867651.2004.10487596 (cit. on pp. 7, 8, 13, 25).

[Wan+23] Qile Wang et al. “A scoping review of the use of lab streaming layer framework in virtual and augmented reality research”. *Virtual reality : the journal of the Virtual Reality Society* 27.3 (2023), pp. 2195–2210 (cit. on p. 25).

[Zha+22] Yunfan Zhang et al. “InDepth: Real-time Depth Inpainting for Mobile Augmented Reality”. *Proceedings of ACM on interactive, mobile, wearable and ubiquitous technologies* 6.1 (2022), pp. 1–25 (cit. on p. 6).

Online sources

[Met24] Meta. *OpenXR Hand Skeleton in Interaction SDK*. Nov. 7, 2024. URL: https://developers.meta.com/horizon/documentation/unity/unity-isdk-openxr-hand (visited on 01/29/2026) (cit. on p. 23).

[Met25] Meta. *Passthrough Camera API overview*. Dec. 11, 2025. URL: https://developers.meta.com/horizon/documentation/spatial-sdk/spatial-sdk-pca-overview (visited on 01/29/2026) (cit. on p. 21).

[Met26] Meta. *OVRSkeleton Class*. 2026. URL: https://developers.meta.com/horizon/reference/unity/v83/class_o_v_r_skeleton/ (visited on 01/29/2026) (cit. on p. 23).

[Sch24] Sebastian Schöner. *Unity Burst and the kernel theory of video game performance*. Dec. 12, 2024. URL: https://blog.s-schoener.com/2024-12-12-burst-kernel-theory-game-performance/ (visited on 01/29/2026) (cit. on p. 45).

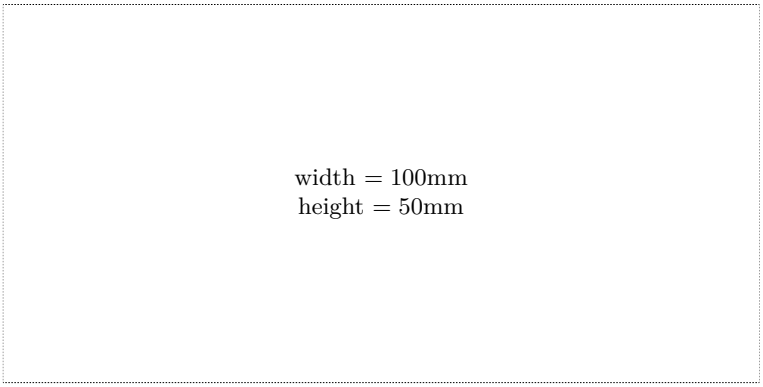
[Sta24] Statista. *Mobile augmented Reality (AR) users worldwide from 2023 to 2028*. 2024. URL: https://www.statista.com/statistics/1098630/global-mobile-augmented-reality-ar-users/ (visited on 08/31/2025) (cit. on p. 1).

[Tec26a] Unity Technologies. *Csharp compiler and language version reference*. Jan. 28, 2026. URL: https://docs.unity3d.com/Manual/csharp-compiler.html (visited on 01/29/2026) (cit. on p. 21).

[Tec26b] Unity Technologies. *Sentis overview*. 2026. URL: https://docs.unity3d.com/Packages/com.unity.ai.inference@2.5/manual/index.html (visited on 01/29/2026) (cit. on p. 29).

Check Final Print Size

— Check final print size! —



— Remove this page after printing! —